IEORE4004 Optimization Models and Methods

Yaren Bilge Kaya, PhD

Columbia University
Department of Industrial Engineering and Operations Research

Outline

- Modeling Piecewise Linear Functions
- Introduction to Solving Integer Programs (IPs)
  - Branch and Bound Algorithm
  - Cutting Planes Algorithm
  - Implementation of Algorithms Using Gurobi

Modeling Fixed & Piecewise Costs

Modeling Fixed Costs

In a typical production planning problem involving N products, the production cost for product j may consist of:

- a fixed cost d_j independent of the amount produced, and
- a variable cost c_j per unit

Thus, if x_j is the production level of product j, its production cost function may be written as:

$$f_j(x_j) = \begin{cases} d_j + c_j x_j, & x_j > 0 \\ 0, & x_j = 0 \end{cases}$$

Modeling Fixed Costs

In a typical production planning problem involving N products, the production cost for product j may consist of:

- a fixed cost d_j independent of the amount produced, and
- a variable cost c_j per unit

Thus, if x_j is the production level of product j, its production cost function may be written as:

$$f_j(x_j) = \begin{cases} d_j + c_j x_j, & x_j > 0 \\ 0, & x_j = 0 \end{cases}$$

And the objective will become: $Min\ Z = \sum_{j \in N} f_j(x_j)$

Modeling Fixed Costs

If it is known that $0 \leq x_j \leq u_j$ and $d_j > 0$ then we can define a binary variable y_j that indicates when the fixed cost is incurred, so that:

$$y_j = 1 \;\; if \; x_j > 0$$
$$y_j = 0 \; if \; x_j = 0$$

Then the contribution to cost due to x_j may be written as:

$$f_j(x_j) = d_j y_j + c_j x_j$$

with the constraints:

Modeling Fixed Costs

If it is known that 0 ≤ x_j ≤ u_j and $d_j > 0$ then we can define a binary variable y_j that indicates when the fixed cost is incurred, so that:

$$y_j = 1 \ \ if \ x_j > 0$$
$$y_j = 0 \ if \ x_j = 0$$

Then the contribution to cost due to x_j may be written as:

$$f_j(x_j) = d_j y_j + c_j x_j$$

Modeling Fixed Costs

If it is known that $0 \leq x_j \leq u_j$ and $d_j > 0$ then we can define a binary variable y_j that indicates when the fixed cost is incurred, so that:

$$y_j = 1 \;\; if \; x_j > 0$$
$$y_j = 0 \; if \; x_j = 0$$

Then the contribution to cost due to x_j may be written as:

$$f_j(x_j) = d_j y_j + c_j x_j$$

with the constraints:

$$x_j \leq \; u_j y_j \; ,$$
$$x_j \geq \epsilon y_j, \qquad \textit{where } \epsilon \textit{ is a very small number}$$
$$x_j \geq 0,$$
$$y \; = \; 0 \; or \; 1.$$

Example: Mobile Care Foundation

MCF aims to control asthma in schoolchildren from low-income families in the greater Chicago region through asthma vans that visit elementary schools on weekdays.

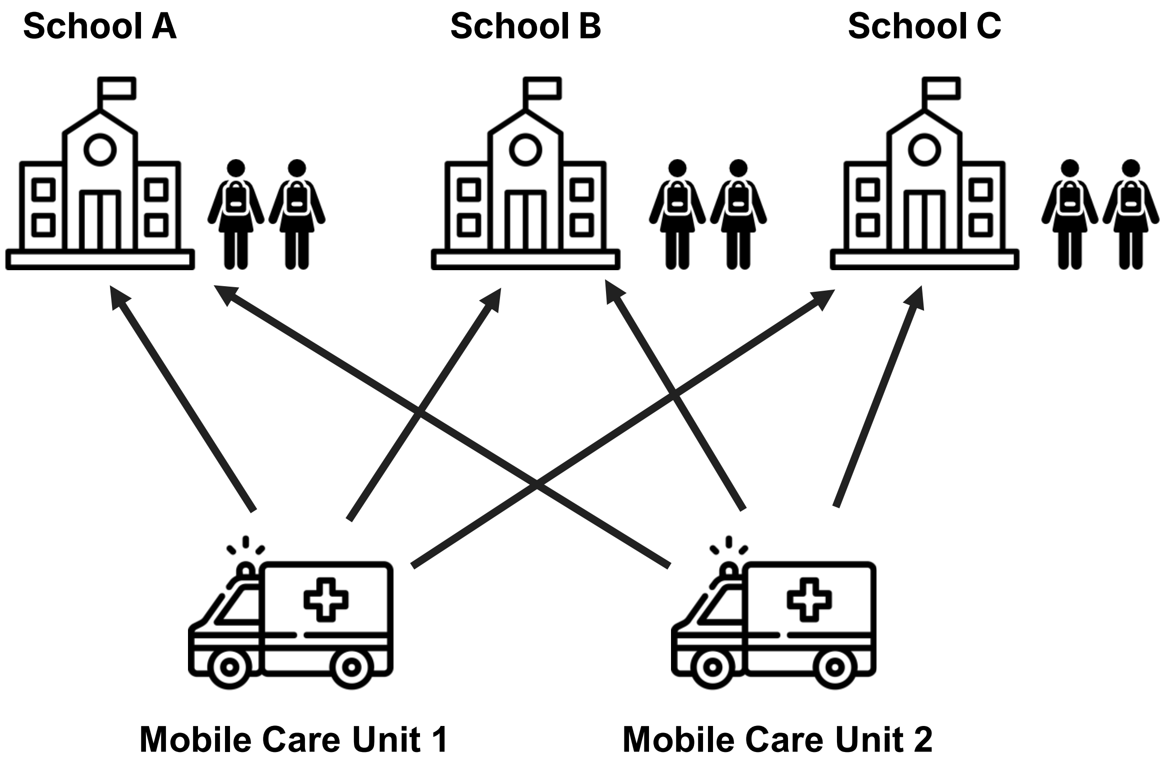
Example: Mobile Care Foundation

MCF aims to control asthma in schoolchildren from low-income families in the greater Chicago region through asthma vans that visit elementary schools on weekdays.

- Operations involve a complex hierarchy of resource allocation decisions aimed at maximizing MCF’s effectiveness
- Now due to funding shortfall, the management is forced to review the performance metrics and execute improvement initiatives.

MCF Optimization

After scaling down, MCF now has 2 vans and 3 schools to cover.
You want to tell the MCF planners which van should go to which school and how many appointments they should cover for one day.
Note that at least 60% of the demand must be covered at each school.

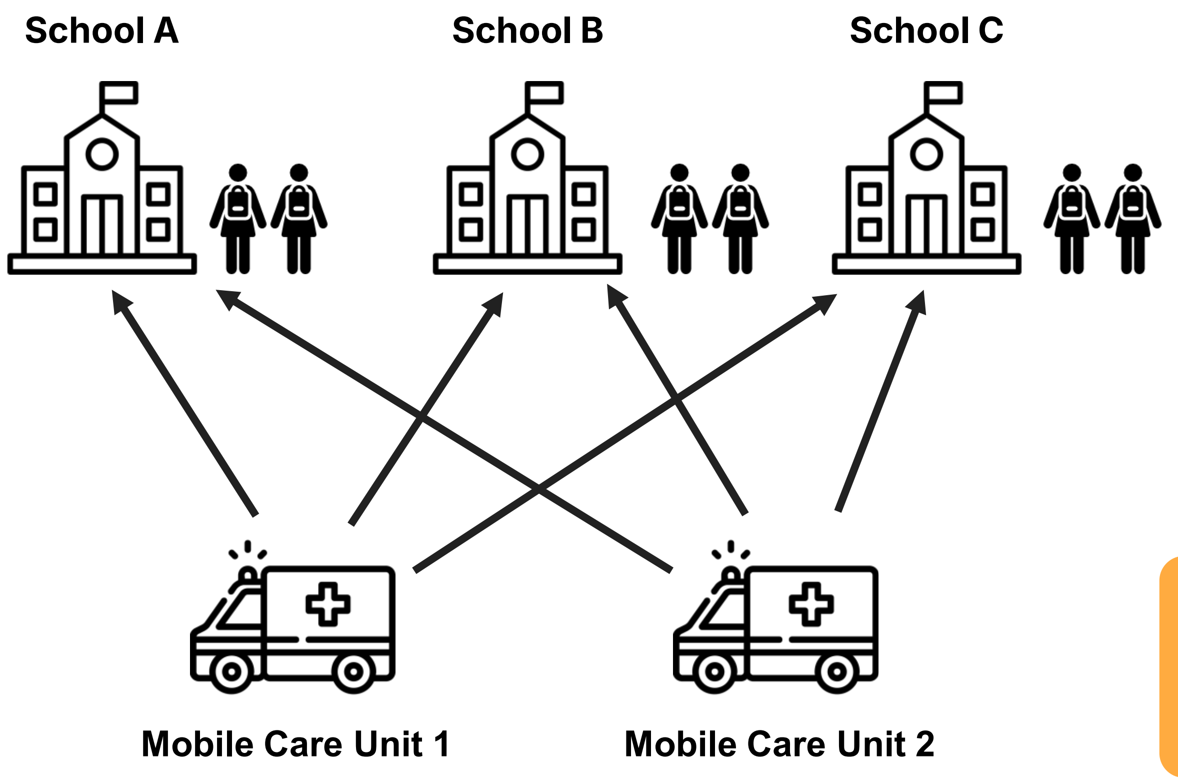


	Van 1	Van 2
Fixed cost	\$40	\$60
Appointment cost	\$5	\$8
Capacity (appts.)	30	80

School	Demand for Appointments
A	40
B	50
C	35

MCF Optimization

After scaling down, MCF now has 2 vans and 3 schools to cover.
You want to tell the MCF planners which van should go to which school and how many appointments they should cover for one day.
Note that at least 60% of the demand must be covered at each school.



	Van 1	Van 2
Fixed cost	$40	$60
Appointment cost	$5	$8
Capacity (appts.)	30	80

School	Demand for Appointments
A	40
B	50
C	35

? With given information, how would you assign MC units to schools?

MCF Resource Matching

Sets

- Set of schools $S \in \{A, B, C\}$
- Set of vans $V \in \{1,2\}$

MCF Resource Matching

Sets

- Set of schools $S \in \{A, B, C\}$
- Set of vans $V \in \{1,2\}$

Parameters

- Number of appointments for each school $(a_{S_A}, a_{S_B}, a_{S_C})$
- Fixed cost for each van (f_{V_1}, f_{V_2})
- Appointment cost for each van (g_{V_1}, g_{V_2})
- Capacity for each van (c_{V_1}, c_{V_2})

MCF Resource Matching

Sets

- Set of schools $S \in \{A, B, C\}$
- Set of vans $V \in \{1,2\}$

Parameters

- Number of appointments for each school $(a_{S_A}, a_{S_B}, a_{S_C})$
- Fixed cost for each van (f_{V_1}, f_{V_2})
- Appointment cost for each van (g_{V_1}, g_{V_2})
- Capacity for each van (c_{V_1}, c_{V_2})

Variables

- $x_{[school,van]}$: Number of appointments assigned to a school by a van
  $(x_{A,1}, x_{A,2}, x_{B,1}, x_{B,2}, x_{C,1}, x_{C,2})$
- $y_{[school,van]}$: if a school is assigned to a van, 1; otherwise, 0
  $(y_{A,1}, y_{A,2}, y_{B,1}, y_{B,2}, y_{C,1}, y_{C,2})$

MCF Resource Matching

Objective

MCF Resource Matching

Objective

1. Minimize the assignment cost

Total Cost = Fixed cost + Variable appointment cost

- Fixed: $40 \cdot (y_{A,1} + y_{B,1} + y_{C,1}) + 60 \cdot (y_{A,2} + y_{B,2} + y_{C,2})$
- Appt.: $5 \cdot (x_{A,1} + x_{B,1} + x_{C,1}) + 8 \cdot (x_{A,2} + x_{B,2} + x_{C,2})$

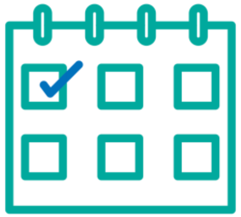
MCF Resource Matching

Objective

1. Minimize the assignment cost

Total Cost = Fixed cost + Variable appointment cost

- Fixed: $40 \cdot (y_{A,1} + y_{B,1} + y_{C,1}) + 60 \cdot (y_{A,2} + y_{B,2} + y_{C,2})$
- Appt.: $5 \cdot (x_{A,1} + x_{B,1} + x_{C,1}) + 8 \cdot (x_{A,2} + x_{B,2} + x_{C,2})$



2. Maximize the number of appointments

Number of appointments:

- $x_{A,1} + x_{B,1} + x_{C,1} + x_{A,2} + x_{B,2} + x_{C,2}$

MCF Resource Matching

Constraints

MCF Resource Matching

Constraints

1. **Cover at least 60% of the demand for appointments at each school**
   - School 1: $x_{A,1} + x_{A,2} \geq 40 \cdot 60\%$
   - School 2: $x_{B,1} + x_{B,2} \geq 50 \cdot 60\%$
   - School 3: $x_{C,1} + x_{C,2} \geq 35 \cdot 60\%$

MCF Resource Matching

Constraints

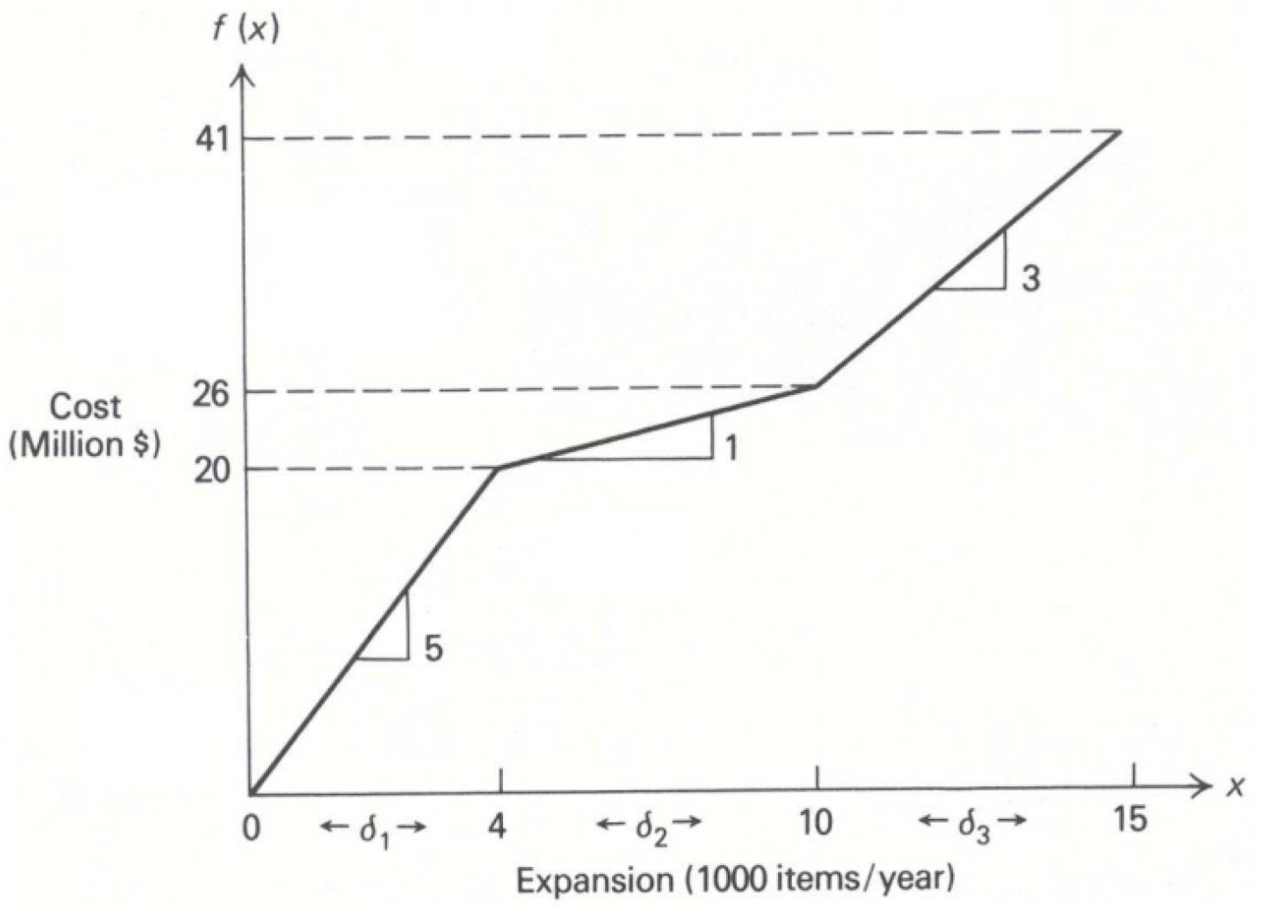
1. **Cover at least 60% of the demand for appointments at each school**
   - School 1: $x_{A,1} + x_{A,2} \geq 40 \cdot 60\%$
   - School 2: $x_{B,1} + x_{B,2} \geq 50 \cdot 60\%$
   - School 3: $x_{C,1} + x_{C,2} \geq 35 \cdot 60\%$
2. **Capacity of a van cannot be exceeded**
   - Van 1: $x_{A,1} + x_{B,1} + x_{C,1} \leq 30$
   - Van 2: $x_{A,2} + x_{A,2} + x_{C,2} \leq 80$

MCF Resource Matching

Constraints

1. **Cover at least 60% of the demand for appointments at each school**
   - School 1: $x_{A,1} + x_{A,2} \geq 40 \cdot 60\%$
   - School 2: $x_{B,1} + x_{B,2} \geq 50 \cdot 60\%$
   - School 3: $x_{C,1} + x_{C,2} \geq 35 \cdot 60\%$
2. **Capacity of a van cannot be exceeded**
   - Van 1: $x_{A,1} + x_{B,1} + x_{C,1} \leq 30$
   - Van 2: $x_{A,2} + x_{A,2} + x_{C,2} \leq 80$
3. **If there are appointments, the van should be assigned to that school**
   - School 1 - Van 1: $x_{A,1} \leq 30 \cdot y_{A,1}$
   - School 1 - Van 2: $x_{A,2} \leq 80 \cdot y_{A,2}$
   - School 2 - Van 1: $x_{B,1} \leq 30 \cdot y_{B,1}$
   - School 2 - Van 2: $x_{B,2} \leq 80 \cdot y_{B,2}$
   - School 3 - Van 1: $x_{C,1} \leq 30 \cdot y_{C,1}$
   - School 3 - Van 2: $x_{C,2} \leq 80 \cdot y_{C,2}$

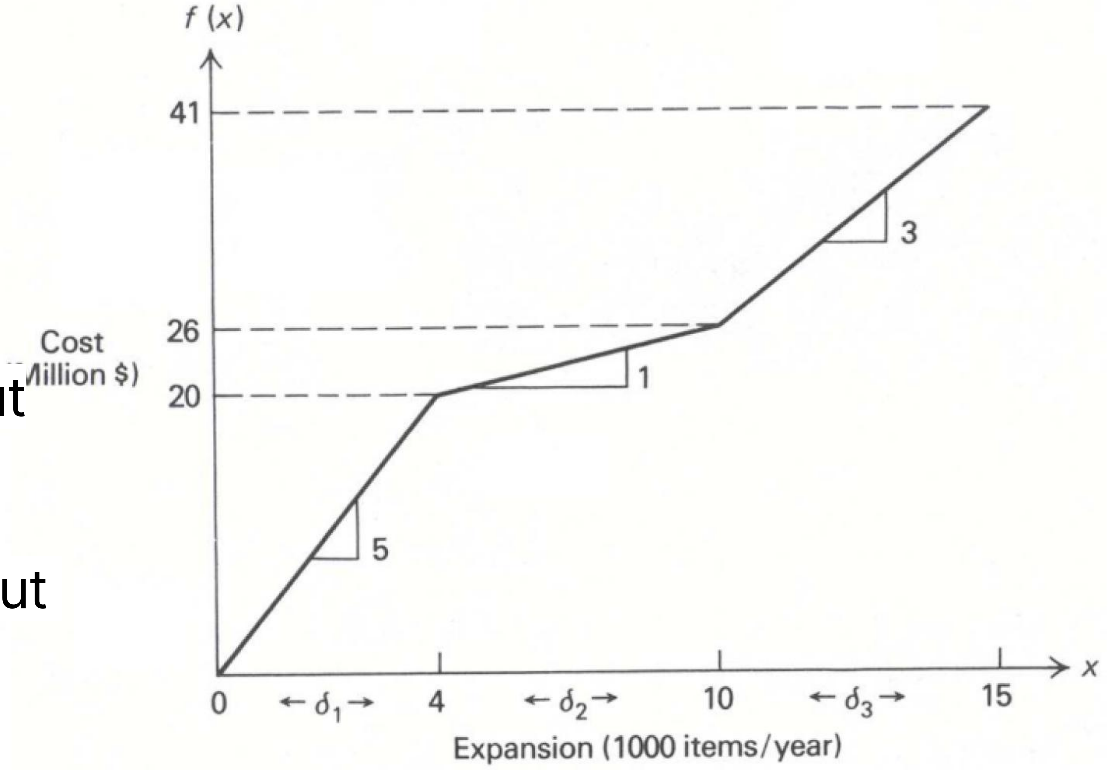
Modeling Piecewise Costs



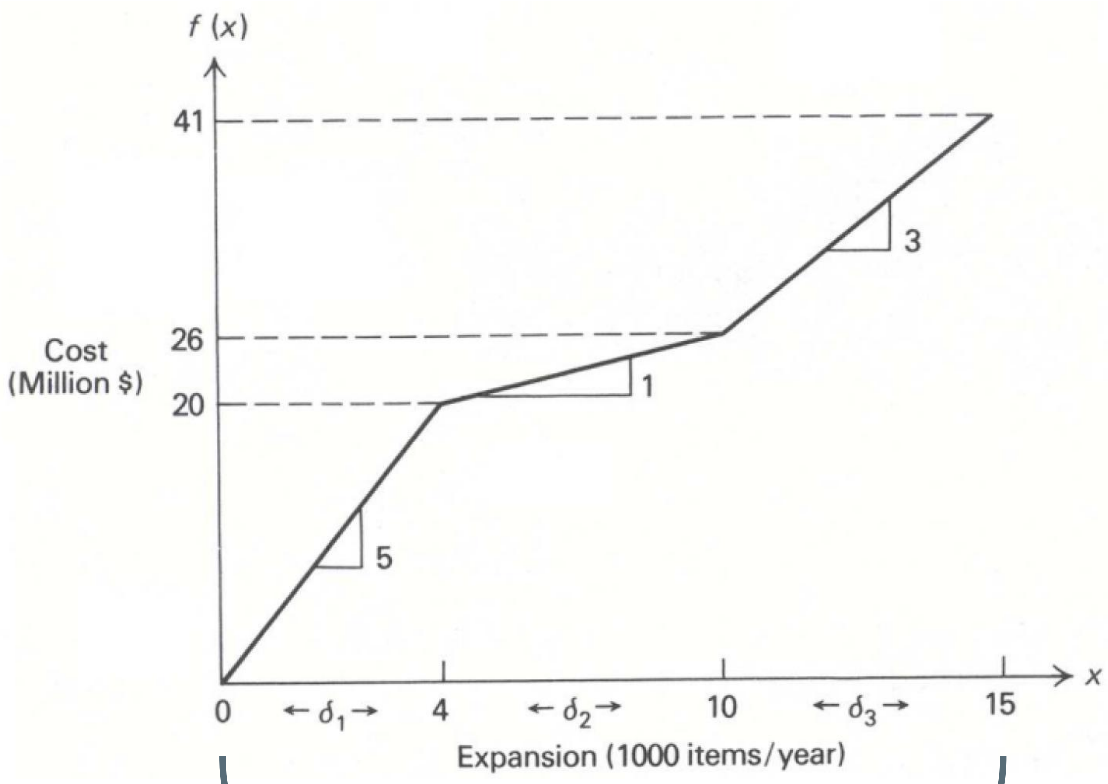
Modeling Piecewise Variable Costs

Define integral variables $\delta_1, \delta_2, and\ \delta_3,$ so tha δ_1

- O1 corresponds to the amount by which x δ_2 :eeds 0, but is less than or equal to 4;
- 02 is the amount by which x exceeds 4, but is l δ_3 than or equal to 10; and
- 03 is the amount by which x exceeds 10, but is less than or equal to 15



Modeling Piecewise Costs



$$x = \delta_1 + \delta_2 + \delta_3$$

where: $x = \delta_1 + \delta_2 + \delta_3$

$$0 \le \delta_1 \le 4,$$
$$0 \le \delta_2 \le 6,$$
$$0 \le \delta_3 \le 5$$

and the total variable cost is given by:

Cost = $5\delta_1 + \delta_2 + 3\delta_3$

Modeling Piecewise Costs

However, we should have the following conditional constraints:

$$\delta_1 = 4, \qquad if\ \delta_2 > 0$$
$$\delta_2 = 6, \qquad if\ \delta_3 > 0$$

Hence, $x = \delta_1 + \delta_2 + \delta_3$

where:

$$0 \le \delta_1 \le 4,$$
$$0 \le \delta_2 \le 6,$$
$$0 \le \delta_3 \le 5$$

and the **total variable cost** is given by:

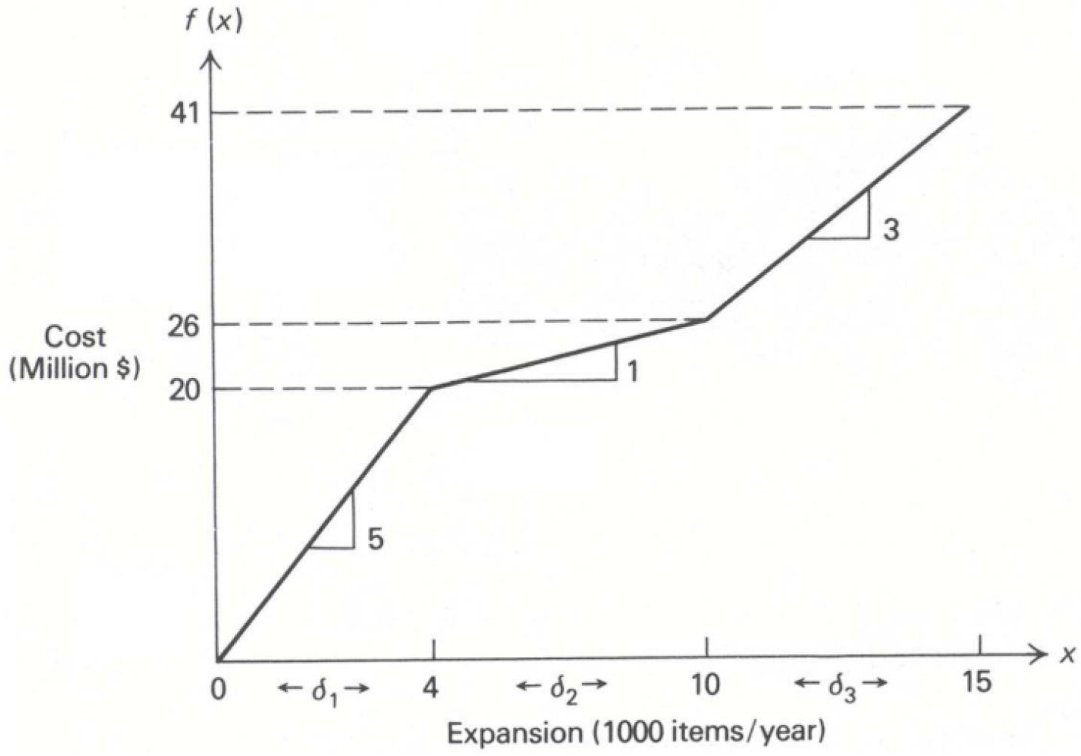
$$Cost = 5\delta_1 + \delta_2 + 3\delta_3$$

Modeling Piecewise Costs

Define

$$w_1 = \begin{cases} 1 & \text{if } \delta_1 \text{ is at its upper bound} \\ 0 & \text{otherwise} \end{cases}$$

$$w_2 = \begin{cases} 1 & \text{if } \delta_2 \text{ is at its upper bound} \\ 0 & \text{otherwise} \end{cases}$$



Modeling Piecewise Costs

Define $w_1 = \begin{cases} 1 & \text{if } \delta_1 \text{ is at its upper bound} \\ 0 & \text{otherwise} \end{cases}$

$w_2 = \begin{cases} 1 & \text{if } \delta_2 \text{ is at its upper bound} \\ 0 & \text{otherwise} \end{cases}$

Then these constraints can be represented as:

$$0 \leq \delta_1 \leq 4, \qquad 0 \leq \delta_2 \leq 6, \qquad 0 \leq \delta_3 \leq 5$$

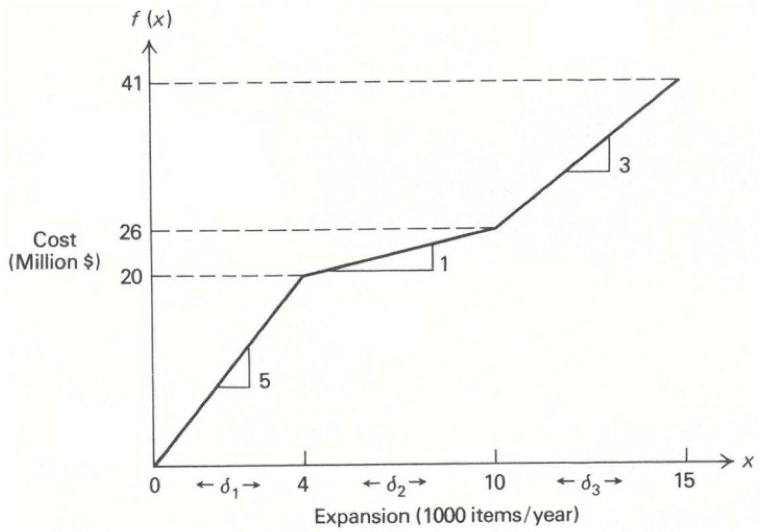
$$\delta_1 = 4, \qquad if\ \delta_2 > 0$$
$$\delta_2 = 6, \qquad if\ \delta_3 > 0$$

Modeling Piecewise Costs

Define $w_1 = \begin{cases} 1 & \text{if } \delta_1 \text{ is at its upper bound} \\ 0 & \text{otherwise} \end{cases}$

$w_2 = \begin{cases} 1 & \text{if } \delta_2 \text{ is at its upper bound} \\ 0 & \text{otherwise} \end{cases}$

$w_1 = 0, w_2 = 0$	corresponding to $0 \le x \le 4$
$w_1 = 1, w_2 = 0$	corresponding to $4 \le x \le 10$
$w_1 = 1, w_2 = 1$	corresponding to $10 \le x \le 15$



Then these constraints can be represented as:

$$0 \le \delta_1 \le 4, \qquad 0 \le \delta_2 \le 6, \qquad 0 \le \delta_3 \le 5$$

$$\delta_1 = 4, \qquad if\ \delta_2 > 0$$
$$\delta_2 = 6, \qquad if\ \delta_3 > 0$$

$$\begin{aligned} 4w_1 &\le \delta_1 \le 4, \\ 6w_2 &\le \delta_2 \le 6w_1, \\ 0 &\le \delta_3 \le 5w_2 \end{aligned}$$

w_1 and w_2 binary.

Example | Supermarket Promotions

Supermarkets routinely run promotions to increase sales

Promotions can:

- Attract new customers
- Increase short-term demand

But too many promotions can:

- Hurt brand perception
- Reduce long-term profitability by hurting the sales of similar items

Example | Supermarket Promotions

In Cohen and Perakis (2020), a model is proposed where the demand depends on the current price and price in the two previous weeks.

Prices for the **13 weeks** in a quarter are chosen to **optimize profit** given this demand.

Other considerations are captured through the following constraints, taken from actual supermarket operations:

- No more than four promotions of an item in a quarter.
- Promotions may not be used in two consecutive weeks.

Example | Supermarket Promotions

- Consider a simpler model where demand $d(t, p(t))$ in week t depends only on the current price $p(t)$ and t.
- Assume all demand is met, so $d(t, p(t))$ is the quantity sold and let c be the unit cost.
- Then the profit contribution of this item over the quarter is:

$$\sum_{t=1}^{13} (p(t) - c)\, d(t, p(t))$$

Note that profit is a nonlinear function of price. However, since it is selected from a small set of allowed prices, we can linearize the integer program.

Suppose we allow four prices: 100%, 90%, 70%, and 60% of p_0 where p_0 is full price and the others are markdowns. Let $d_t(p_k)$ be the demand in week t if the k^{th} price is used.

Example | Supermarket Promotions

- The decision variables are:

$$x_{tk} = \begin{cases} 1 & \text{if the } k\text{th price is used in week } t \\ 0 & \text{otherwise} \end{cases}$$

In other words, $x_{tk} = 1$ if the price in week t is p_k .

Example | Supermarket Promotions

The IP to maximize weekly profit is:

$$\max \sum_{t=1}^{13} \sum_{k=1}^{3} (p_k - c)\, d_t(p_k)\, x_{tk}$$

$$s.t. \quad \sum_{k=1}^{3} x_{tk} = 1, \quad t = 1, \ldots, 13$$

$$\sum_{t=1}^{13} \sum_{k=2}^{3} x_{tk} \leq 4$$

$$\sum_{k=2}^{3} x_{tk} + \sum_{k=2}^{3} x_{t+1,k} \leq 1, \quad t = 1, \ldots, 12$$

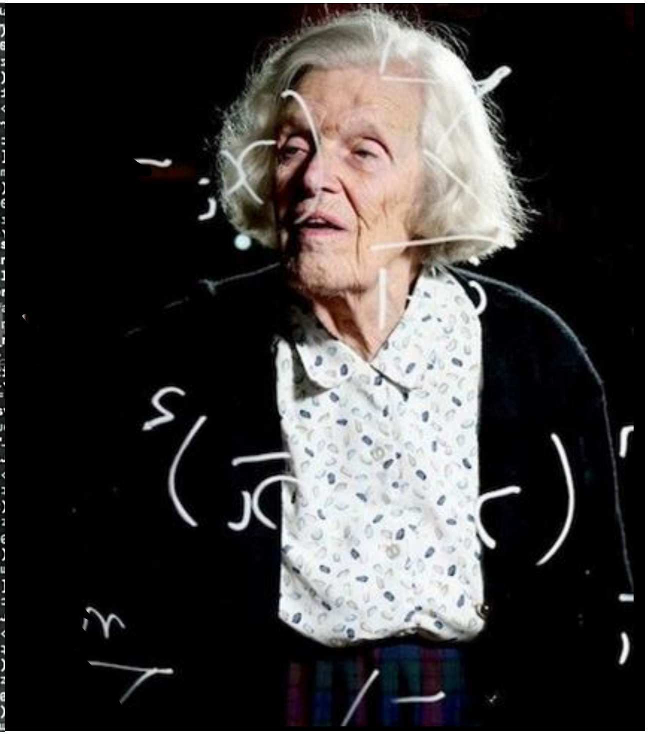
Introduction to Solving Integer Programs

- Branch and Bound Algorithm
- Cutting Planes Algorithm
- Implementation of Algorithms Using Gurobi

Branch and Bound Algorithm

Branch and Bound Algorithm

The method was first proposed by Ailsa Land and Alison Harcourt (Doig) whilst carrying out research at the London School of Economics in 1960 for discrete programming



[A special note from Alison](#) *to women in STEM*

Branch and Bound Algorithm

Let's consider this IP to explain the method:

$$\text{Maximize } Z = 5x_1 + 6x_2$$

$$\text{Subject to} \quad x_1 + x_2 \le 5$$

$$4x_1 + 7x_2 \le 28$$

$$x_1, x_2 \ge 0$$

$$x_1, x_2 \in Z$$

Branch and Bound Algorithm

Let's consider this IP to explain the method:

$$\begin{aligned} &\textit{Maximize } Z = 5x_1 + 6x_2 \\ &\textit{Subject to} \quad x_1 + x_2 \leq 5 \\ &\qquad 4x_1 + 7x_2 \leq 28 \\ &\qquad x_1, x_2 \geq 0 \\ &\qquad x_1, x_2 \in Z \end{aligned}$$

$$\begin{aligned} &\textit{Maximize } Z = 5x_1 + 6x_2 \\ &\textit{Subject to} \quad x_1 + x_2 \leq 5 \\ &\qquad 4x_1 + 7x_2 \leq 28 \\ &\qquad x_1, x_2 \geq 0 \end{aligned}$$

Step 1: Remove the integrality constraints, a.k.a. Linear Relaxation

Linear Relaxation

The *Linear Relaxation* of an IP is the LP obtained by removing the integrality constraints!

IP

$$\begin{aligned} \text{min } z &= 50x_1 + 100x_2 \\ \text{s.t.} \quad & 7x_1 + 2x_2 \geq 28 \\ & 2x_1 + 12x_2 \geq 24 \\ & x_1, x_2 \geq 0 \\ & x_1, x_2 \in Z \end{aligned}$$

LP

$$\begin{aligned} \text{min } z &= 50x_1 + 100x_2 \\ \text{s.t.} \quad & 7x_1 + 2x_2 \geq 28 \\ & 2x_1 + 12x_2 \geq 24 \\ & x_1, x_2 \geq 0 \end{aligned}$$

Linear Relaxation

Linear Relaxation

The *Linear Relaxation* of an IP is the LP obtained by removing the integrality constraints!

- If the original LP is a **maximization** problem, then the optimal value of the LR gives an **Upper Bound** to the optimal solution of the IP.
- If the original IP is a **minimization** problem, then the optimal value of the LR gives a **Lower Bound** to the optimal solution of the IP.

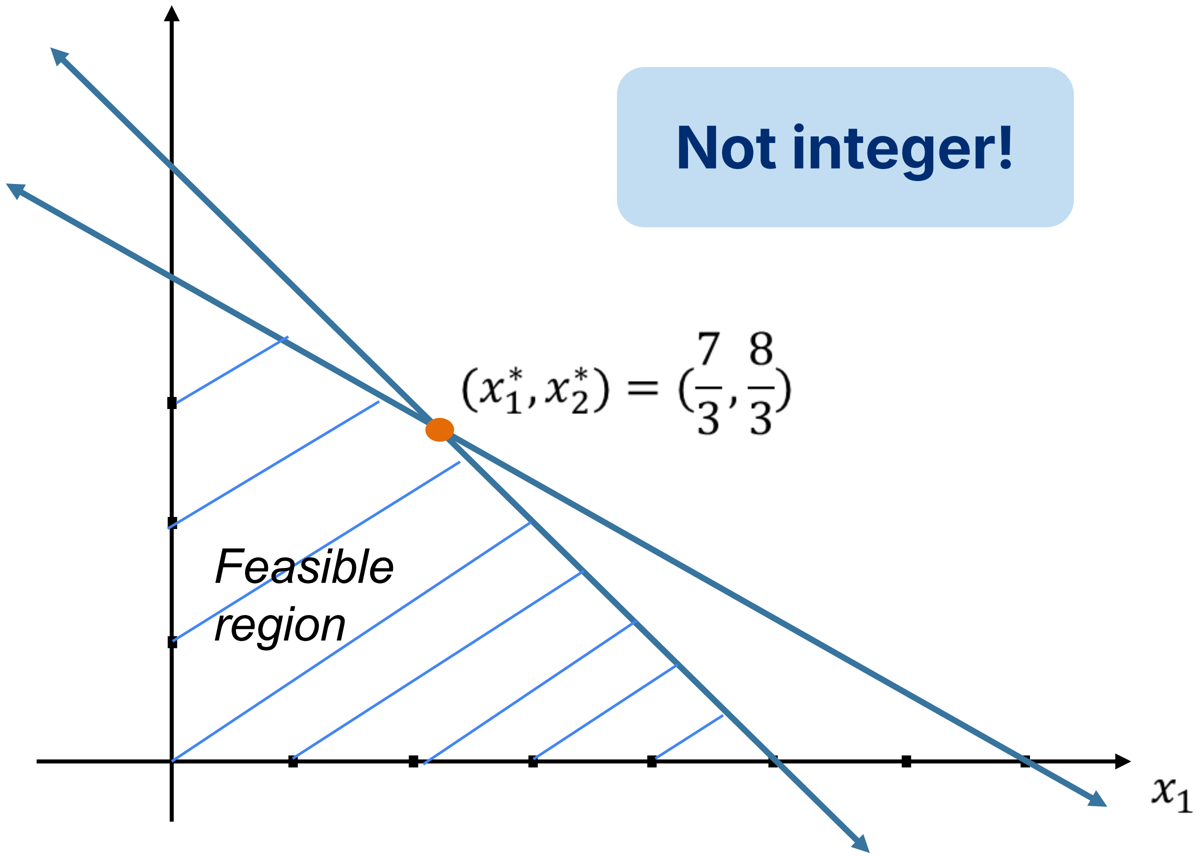
Branch and Bound Algorithm

Step 2: See whether the LP's optimal solution is integer

Maximize $Z = 5x_1 + 6x_2$

Subject to

$$x_1 + x_2 \leq 5$$
$$4x_1 + 7x_2 \leq 28$$
$$x_1, x_2 \geq 0$$
$$x_1, x_2 \in Z$$

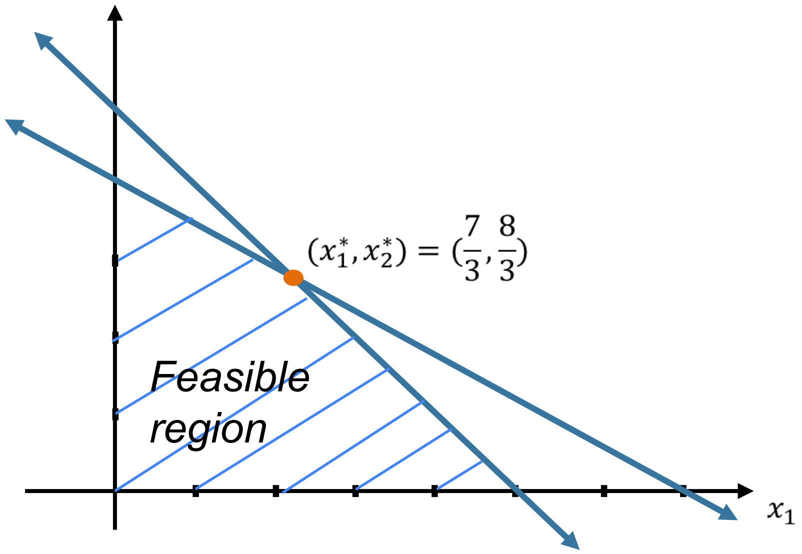


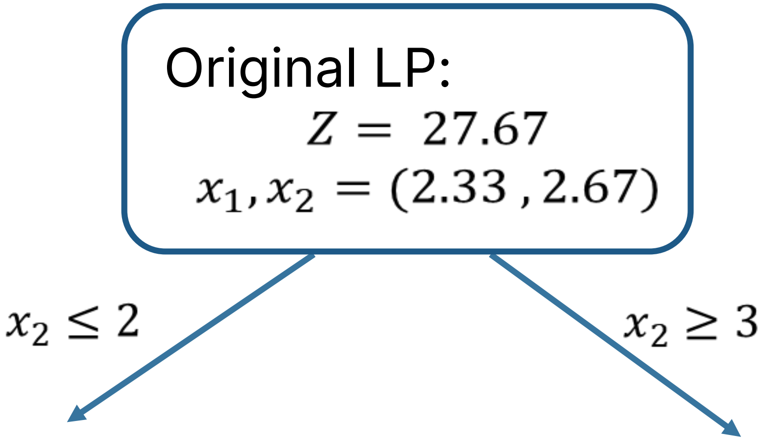
Not integer!

Branch and Bound Algorithm

Step 3: Check the neighboring integer solutions

Pick randomly, or the largest decimal portion, or ...





Branch and Bound Algorithm

Step 3: Check the neighboring integer solutions

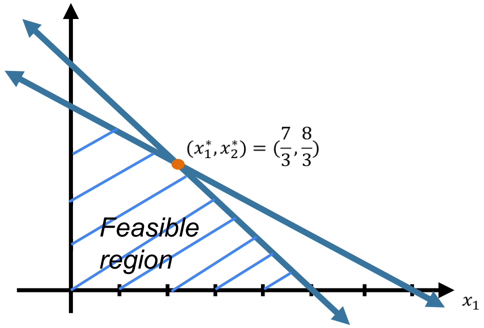
Maximize $Z = 5x_1 + 6x_2$

Subject to $x_1 + x_2 \leq 5$

$4x_1 + 7x_2 \leq 28$

$x_1, x_2 \geq 0$

$x_2 \leq 2$

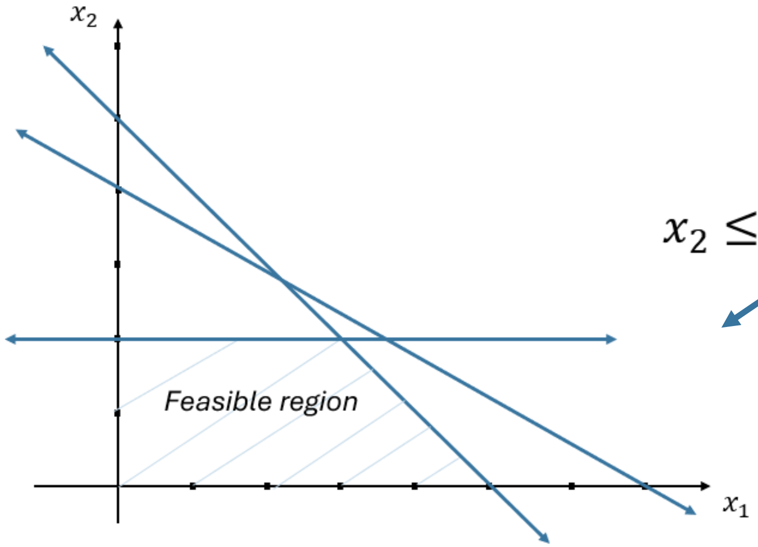


Original LP:

$Z = 27.67$

$x_1, x_2 = (2.33\,, 2.67)$

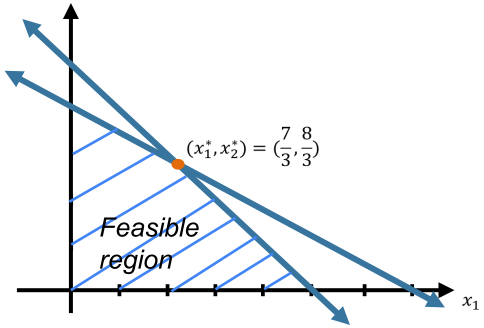
$x_2 \leq 2$



Branch and Bound Algorithm

Step 3: Check the neighboring integer solutions

Maximize $Z = 5x_1 + 6x_2$
Subject to $x_1 + x_2 \leq 5$
$4x_1 + 7x_2 \leq 28$
$x_1, x_2 \geq 0$
$x_2 \leq 2$

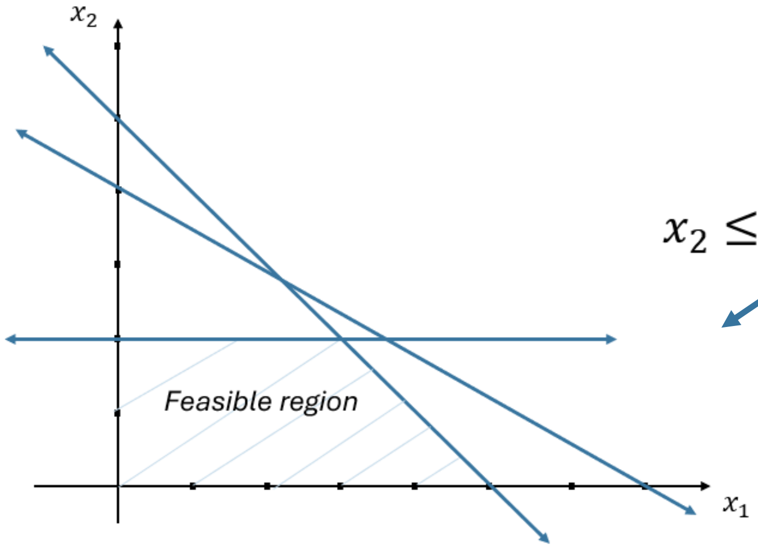


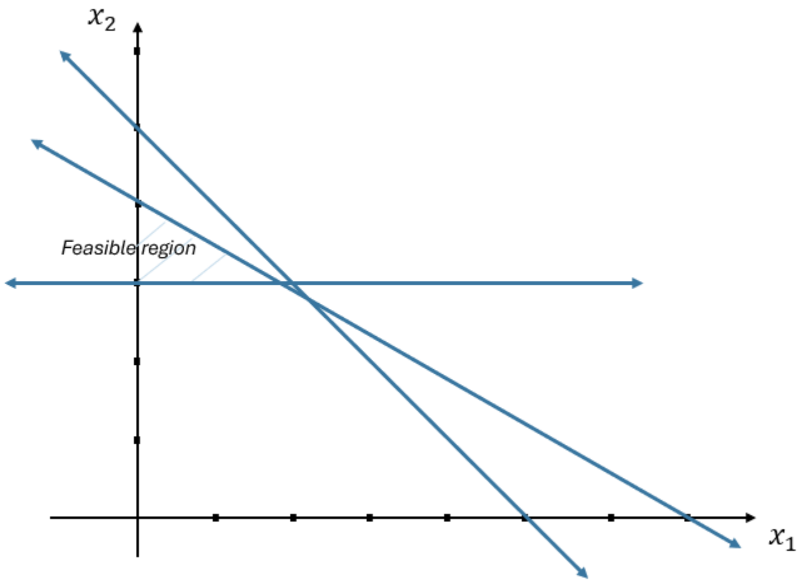
Maximize $Z = 5x_1 + 6x_2$
Subject to $x_1 + x_2 \leq 5$
$4x_1 + 7x_2 \leq 28$
$x_1, x_2 \geq 0$
$x_2 \geq 3$

Original LP:
$Z = 27.67$
$x_1, x_2 = (2.33\,, 2.67)$

$x_2 \leq 2$

$x_2 \geq 3$





Branch and Bound Algorithm

Step 3: Check the neighboring integer solutions

Maximize $Z = 5x_1 + 6x_2$

Subject to

$$x_1 + x_2 \le 5$$
$$4x_1 + 7x_2 \le 28$$
$$x_1, x_2 \ge 0$$
$$x_2 \ge 3$$

Original LP:

$$Z = 27.67$$
$$x_1, x_2 = (2.33, 2.67)$$

$x_2 \ge 3$

Sub-problem 2:

$$Z = 26.75$$
$$x_1, x_2 = (1.75, 3)$$

Branch and Bound Algorithm

Original LP:

$Z = 27.67$

$x_1, x_2 = (2.33, 2.67)$

$x_2 \geq 3$

Sub-problem 2:

$Z = 26.75$

$x_1, x_2 = (1.75, 3)$

$x_1 \leq 1$

$x_1 \geq 2$

Infeasible

Maximize $Z = 5x_1 + 6x_2$

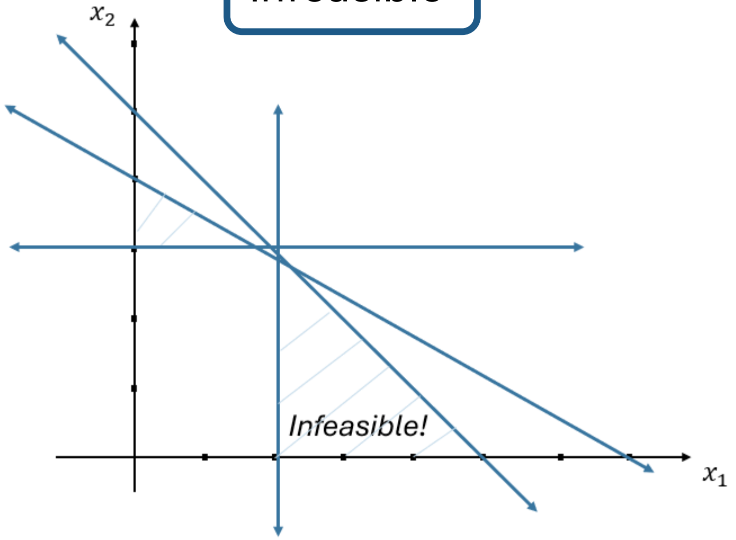
Subject to $x_1 + x_2 \leq 5$

$4x_1 + 7x_2 \leq 28$

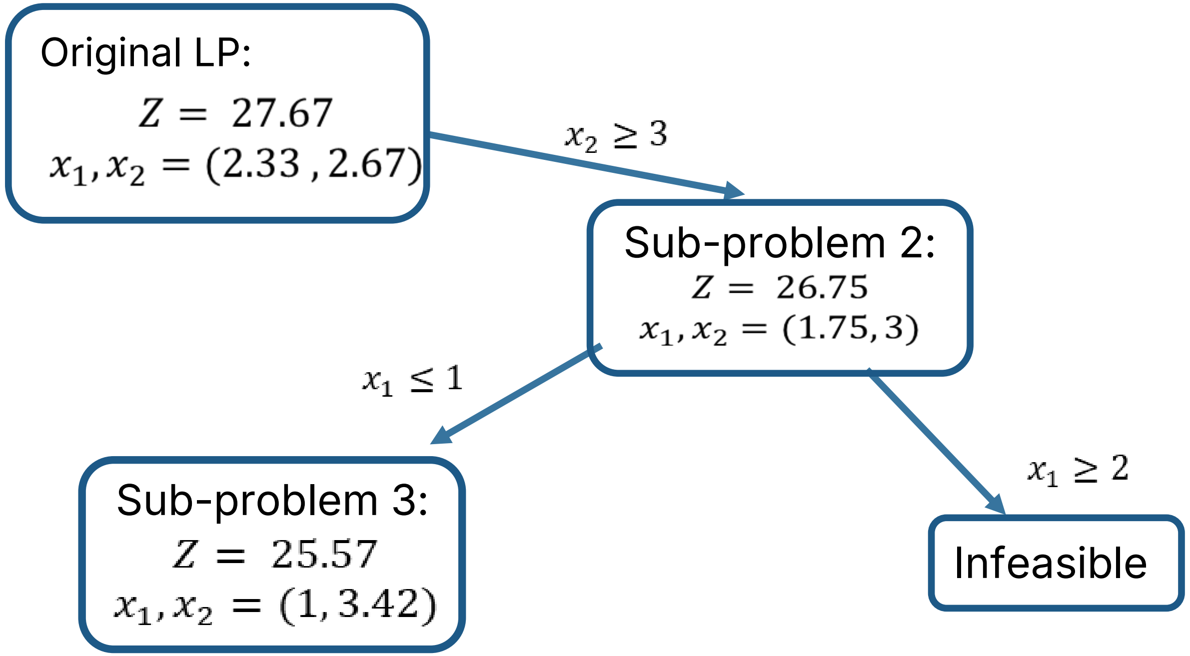
$x_1, x_2 \geq 0$

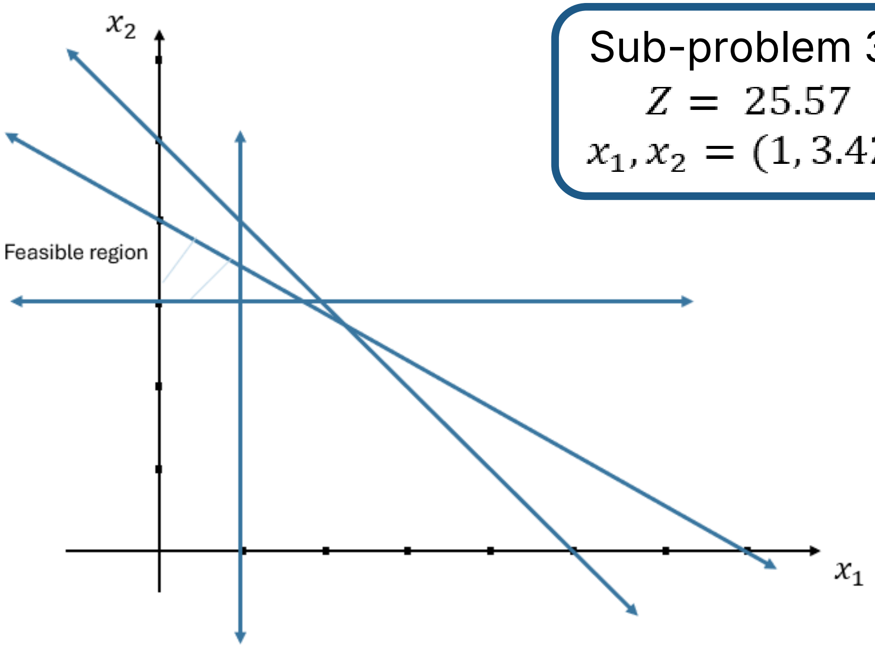
$x_2 \geq 3$

$x_1 \geq 2$

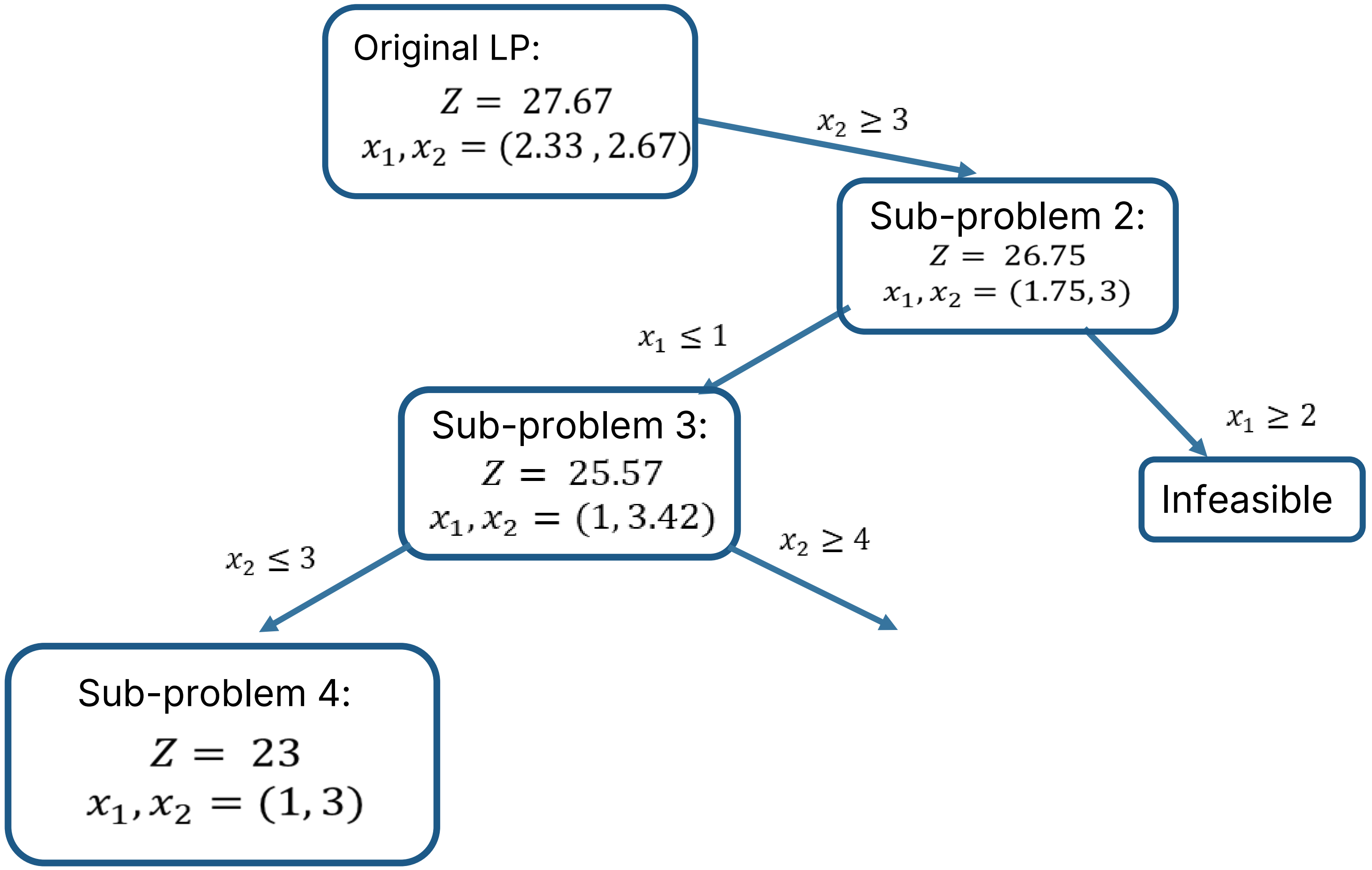


Branch and Bound Algorithm



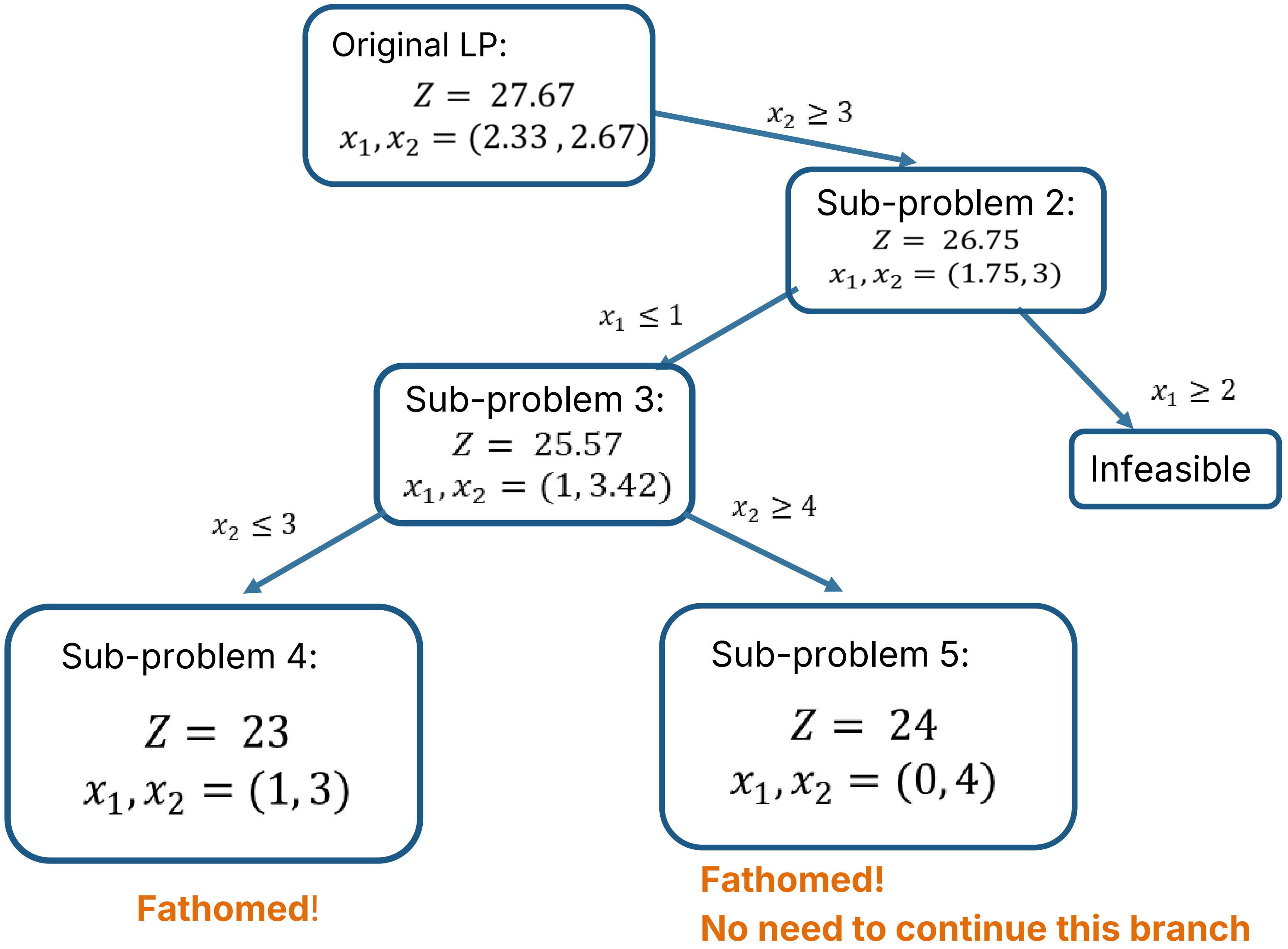


Branch and Bound Algorithm

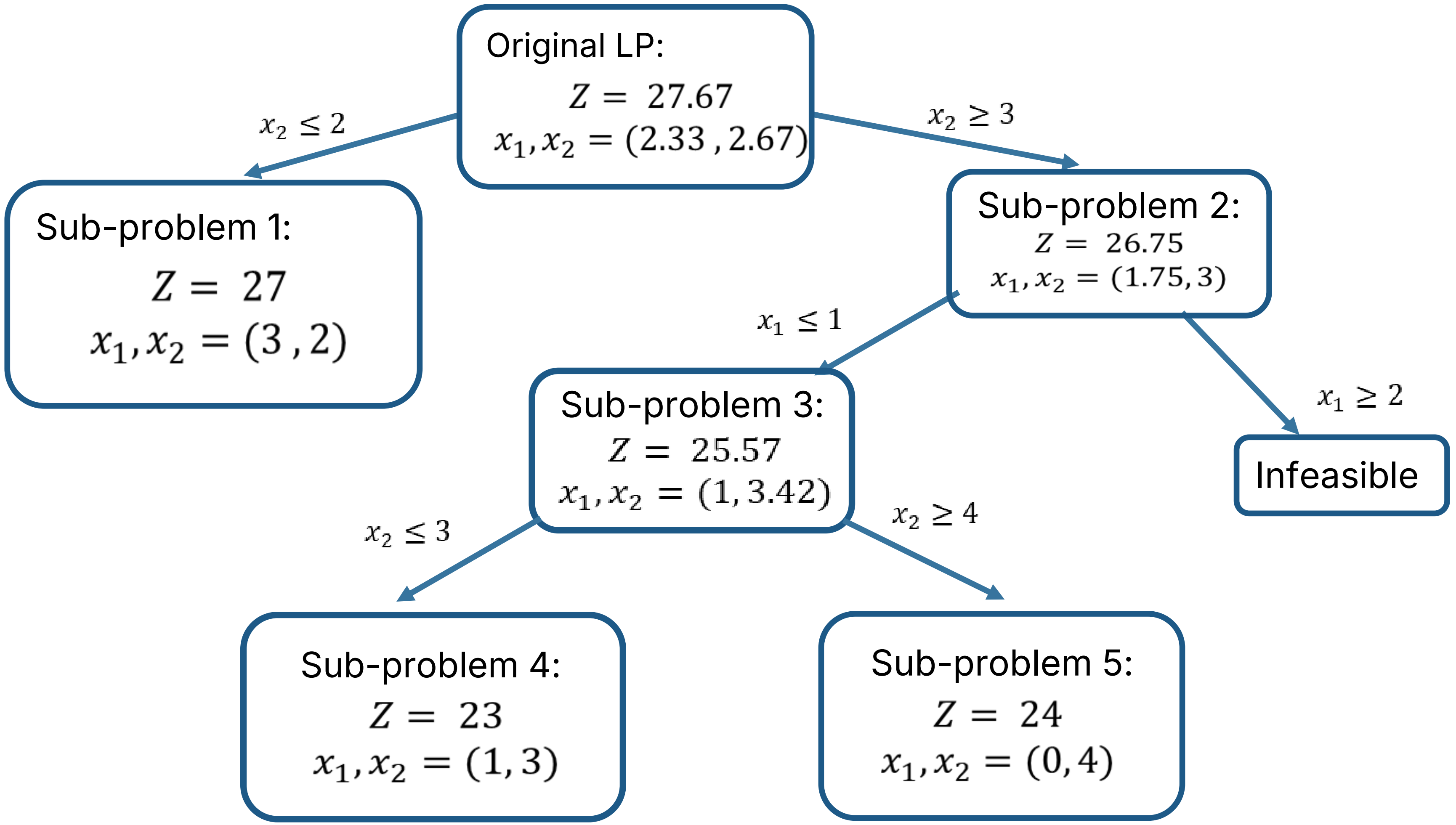


An integer solution, no need to continue this branch - Lower Bound!

Branch and Bound Algorithm



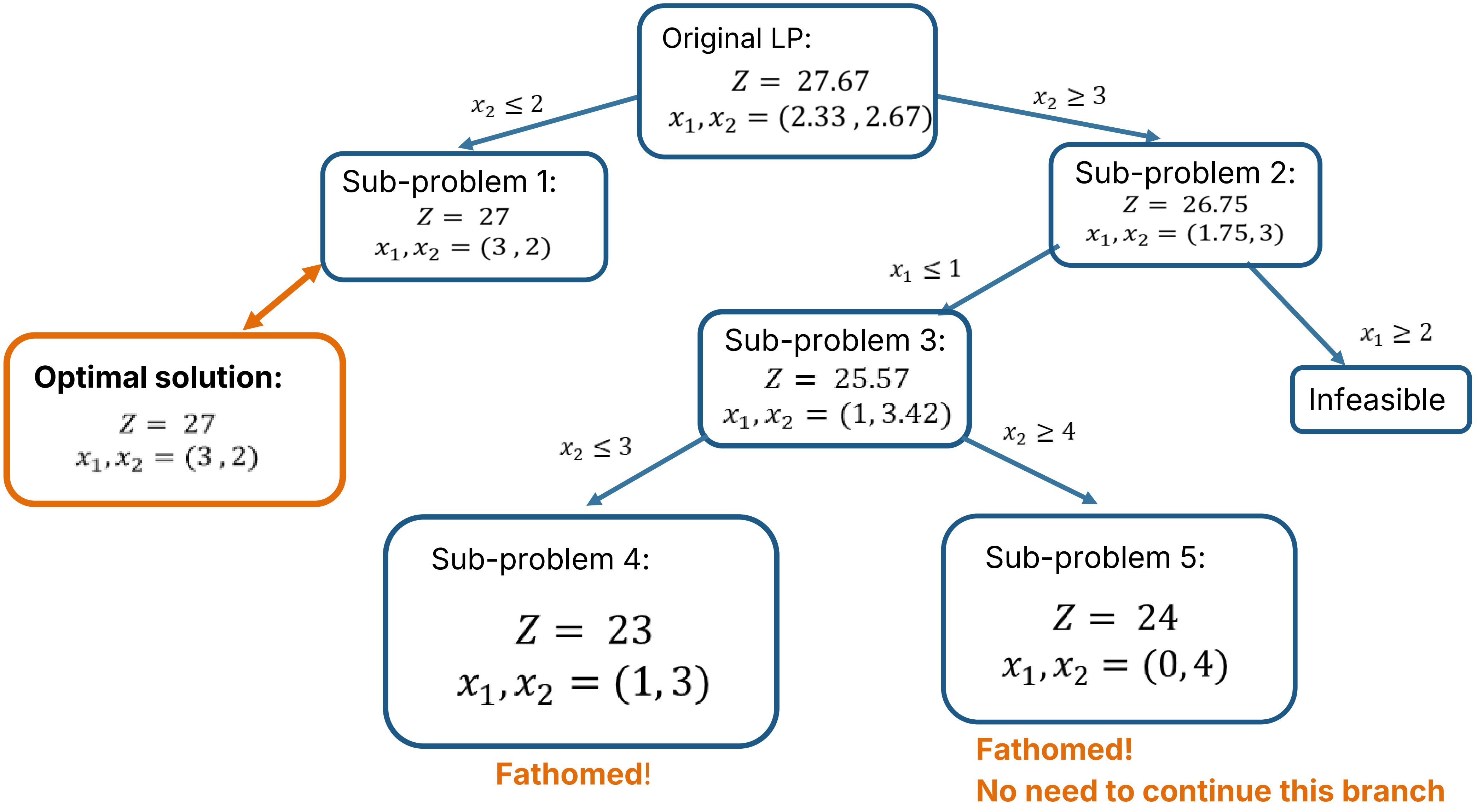
Branch and Bound Algorithm



Fathomed!

Fathomed!
No need to continue this branch

Branch and Bound Algorithm



Branch and Bound Algorithm in a Nutshell

Suppose $\bar{x}$ is the integer solution of maximum profit found so far.

Iteratively consider a subproblem (node) of the branching tree:

Step 1: Solve the LR

Step 2:

- if x^* is integer and profit of x^* is larger than the profit of $\bar{x}$, prune the node and set $\bar{x} = x^*$
- if the profit of x^* is not larger than the profit of $\bar{x}$, prune the node
- if it is infeasible, prune the node
- else, let $x_i^* = \alpha \notin Z$. Create two subproblems adding $x_i \leq \lfloor \alpha \rfloor$ and $x_i \leq \lceil \alpha \rceil$ to the original IP.

What if the problem is Mixed-Integer?

$$\max c^T x + d^T y$$

$$\begin{array}{rcrcl} Ax & + & By & \leq & b \\ x & & & \in & \mathbb{Z}^n \\ & & y & \in & \mathbb{R}^m \end{array}$$

Iteratively decomposes the original problem into two suproblems, storing in $(\bar{x}, \bar{y})$ the best feasible solution found so far.

For each of those subproblems:

- Solve the Linear Relaxation
- If the LR is infeasible, prune the node. Else, let (x^*, y^*) the optimal solution;
- If $x^* \in Z$ and $c^T x^* + d^T y^* > c^T \bar{x} + d^T \bar{y}$, prune the node and set $(\bar{x}, \bar{y}) = (x^*, y^*)$
- If $x^* \notin Z$. If $c^T x^* + d^T y^* \leq c^T \bar{x} + d^T \bar{y}$, prune the node.
- Else, let $x_i^* = \alpha \notin Z$. Create two subproblems adding $x_i \leq \lfloor \alpha \rfloor$ and $x_i \leq \lceil \alpha \rceil$

Pros and Cons of B&B

Usually good in practice.

In general, no guarantee on when it will terminate.

Cutting Planes Algorithm

Integer Programming Formulations

There are often many ways to formulate an IP

- Each way yields the exact same integer program
- The associated linear program relaxations, however, *may be very different*
  - They differ primarily in the bounds they yield

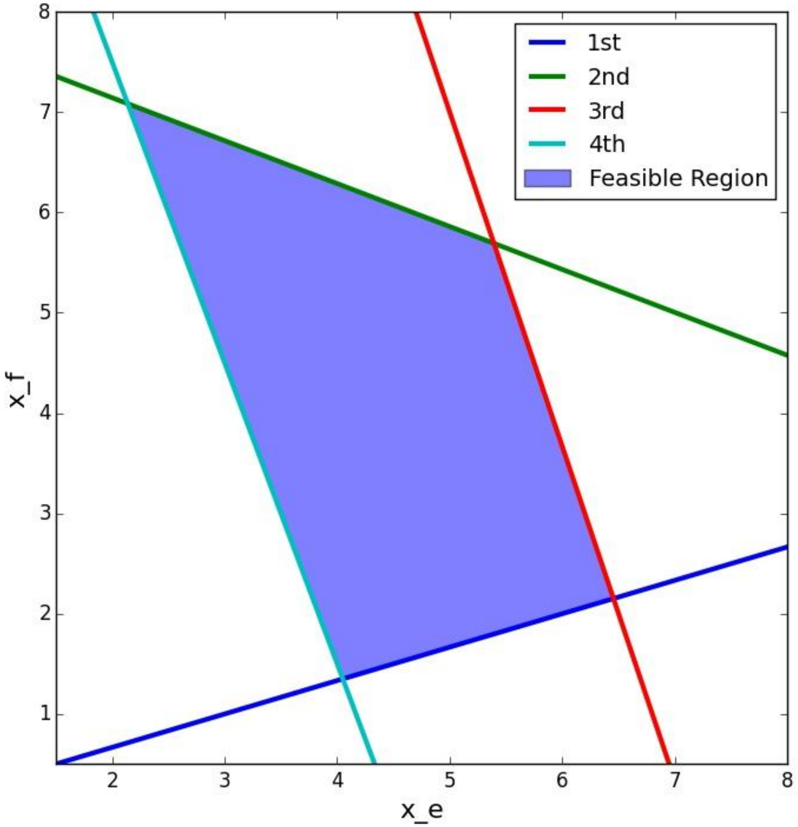
Illustrative IP: LP vs. IP Solutions

Let's consider this IP

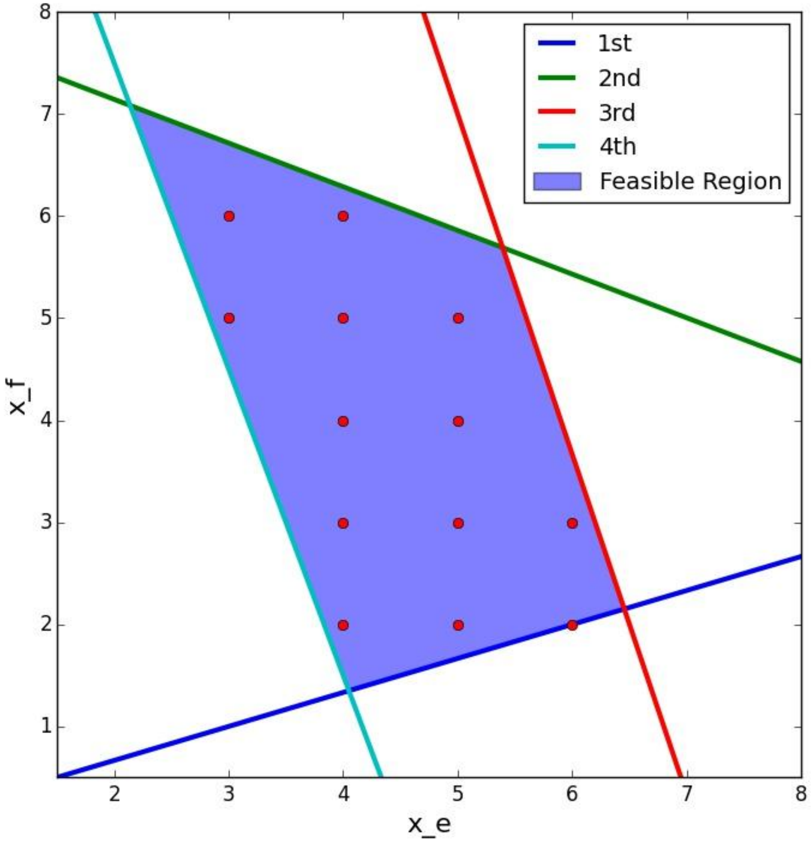
$$
\begin{aligned}
\text{max} \quad & z = 18x_e + 6x_f \\
\text{subject to} \quad & x_e - 3x_f \leq 0 \\
& 42.8x_e + 100x_f \leq 800 \\
& 20x_e + 6x_f \leq 142 \\
& 30x_e + 10x_f \geq 135 \\
& x_e, x_f \geq 0, \ \text{integer}
\end{aligned}
$$

Illustrative IP: LP vs. IP Solutions

It has a feasible region that appears roughly like this:



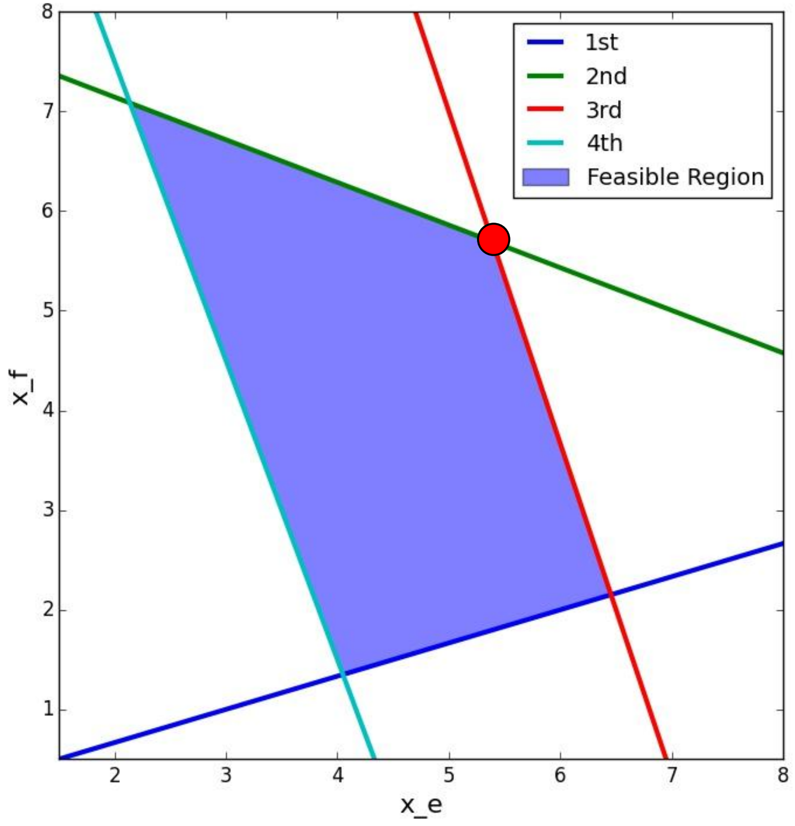
With integer variable restrictions, like this:



Illustrative IP: LP vs. IP Solutions

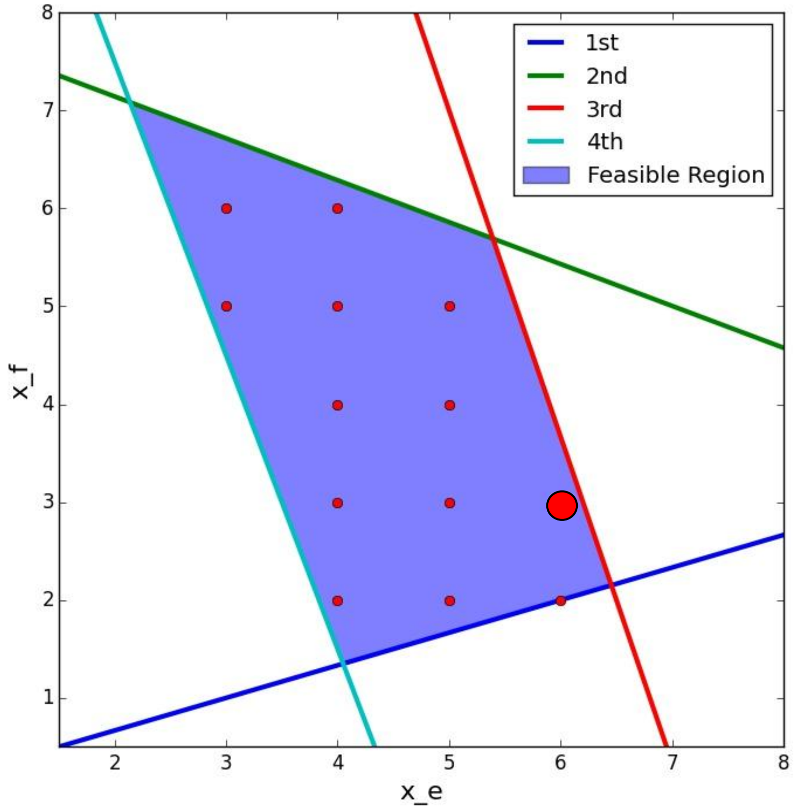
The optimal LP relaxation solution is:

$$z_{LP}^* = 131.2, x_e^* = 5.392, x_f^* = 5.692$$



The optimal integer solution is:

$$z_{IP}^* = 126, x_e^* = 6, x_f^* = 3$$



What's Best We Can Do?

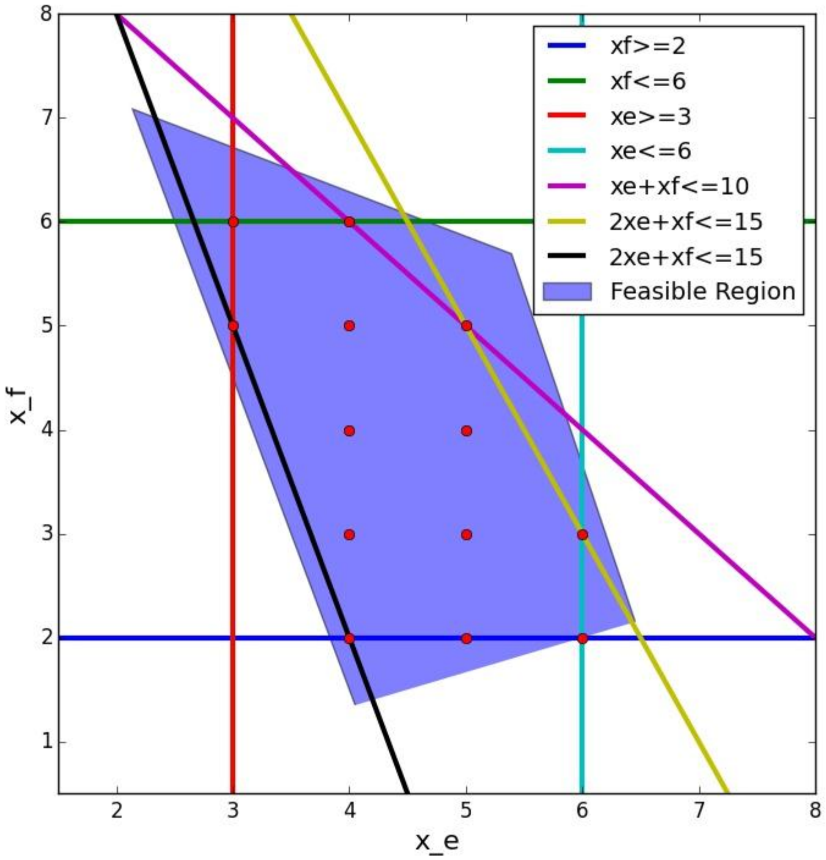
We can add *cutting planes* that eliminate *fractional solutions* to the LP without eliminating any *integer solutions*

What's Best We Can Do?

We can add *cutting planes* that eliminate *fractional solutions* to the LP without eliminating any *integer solutions*

Adding the following 7 constraints, or *cutting planes*, to the problem yields the following:

$$x_f \geq 2$$
$$x_f \leq 6$$
$$x_e \geq 3$$
$$x_e \leq 6$$
$$x_e + x_f \leq 10$$
$$2x_e + x_f \leq 15$$
$$3x_e + x_f \geq 14$$



What's Best We Can Do?

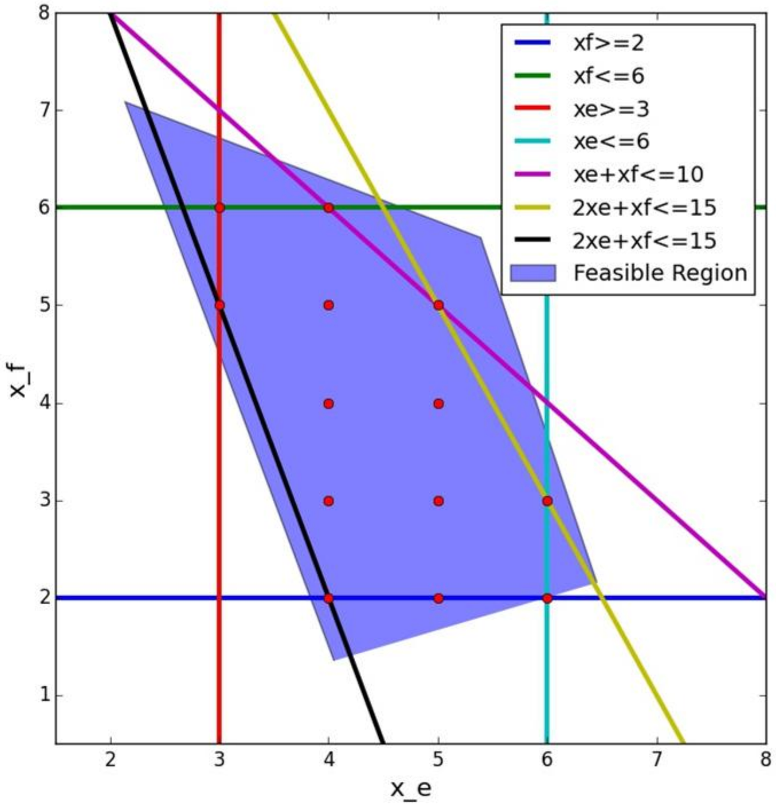
Adding these seven ***valid inequalities*** causes every corner point of the LP to be integer

Thus, the IP can be solved by solving the LP relaxation

What's Best We Can Do?

Adding these seven ***valid inequalities*** causes every corner point of the LP to be integer

Thus, the IP can be solved by solving the LP relaxation



What's Best We Can Do?

Adding these seven ***valid inequalities*** causes every corner point of the LP to be integer

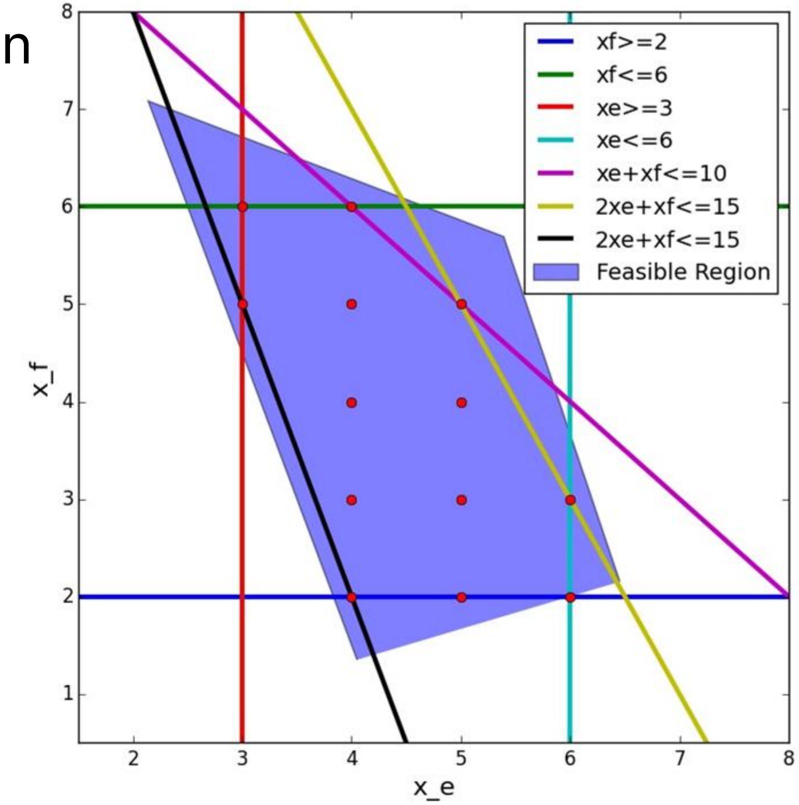
Thus, the IP can be solved by solving the LP relaxation

These 7 cutting planes form the ***convex hull of integer solutions***

- The deepest cuts are called ***facets***

- We obtained it by inspection, but will discuss some general ways to obtain it later in the lecture

- *Such constraints are difficult to find for general (and especially, large) IPs*



How to Find Such Valid Inequalities?

There are approaches to find valid, or even facet-defining, inequalities for integer programs

- Some are problem specific
- Others are general to any integer program

These valid inequalities are typically added during the branch and bound process

- Known as *branch and cut*

General Valid Inequalities

The motivation is rather straightforward

- Consider the following generic integer program:

$$
\begin{array}{rl}
\text{max} & c^\top x \\
\text{subject to} & Ax \leq b \\
& x \geq 0, \text{ integer}
\end{array}
$$

General Valid Inequalities

The motivation is rather straightforward

- Consider the following generic integer program:

$$\begin{aligned} \text{max} \quad & c^\top x \\ \text{subject to} \quad & Ax \le b \\ & x \ge 0, \ \text{integer} \end{aligned}$$

- Now suppose we have any $u \in \mathbb{R}^m_+$
- Then it is entirely valid to write: $(u^\top A)x \le u^\top b$
  - Right?

General Valid Inequalities

- Observe that our original variables are nonnegative: $x \geq 0$
- Then the following operation to the left-hand side of the relationship is valid to do: $(\lfloor u^\top A \rfloor)x \leq u^\top b$
  - *Why is this?*

General Valid Inequalities

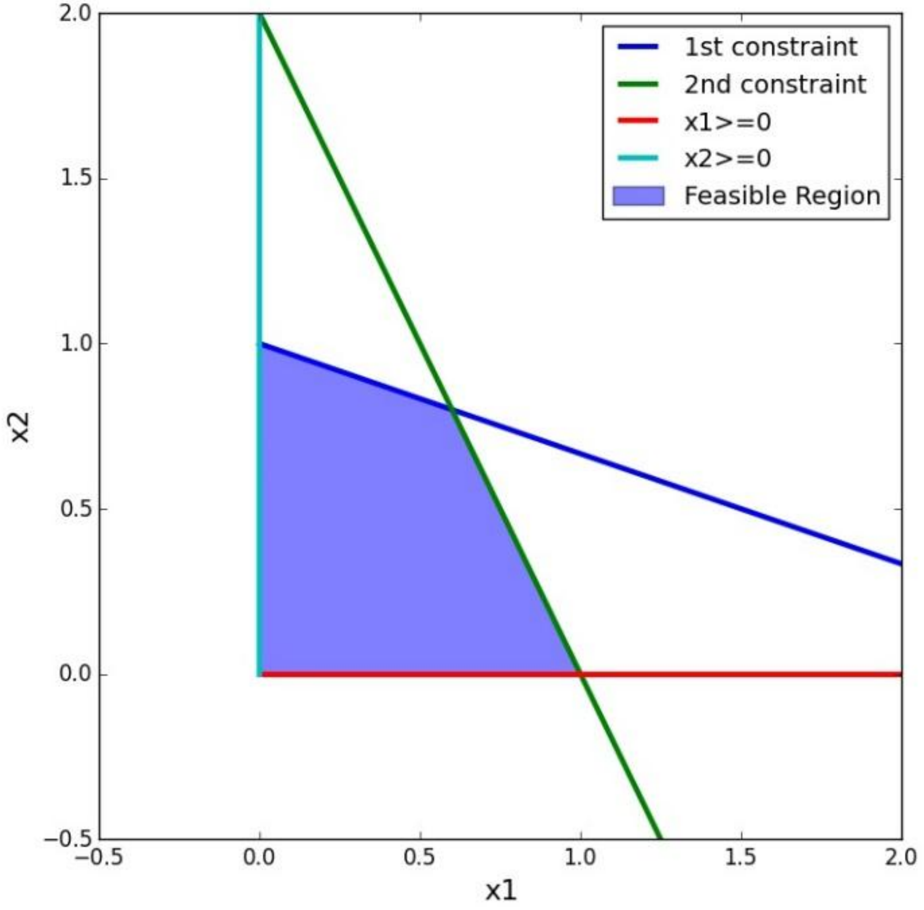
- Observe that our original variables are nonnegative: $x \geq 0$
- Then the following operation to the left-hand side of the relationship is valid to do: $(\lfloor u^\top A \rfloor)x \leq u^\top b$
  - *Why is this?*
- ***Here's the key:***
  - Because of the integrality of both $\lfloor u^\top A \rfloor$ and x
  - We can now write: $(\lfloor u^\top A \rfloor)x \leq \lfloor u^\top b \rfloor$
  - This inequality is known as a ***Chvátal-Gomory*** **(CG)** cut

(The ability to round down the right-hand side is very important)

Example: Chvátal-Gomory Cut

- Suppose we have the following system:

$$\begin{aligned} \text{max} \quad & 5x_1 + 4x_2 \\ \text{subject to} \quad & x_1 + 3x_2 \leq 3 \\ & 2x_1 + x_2 \leq 2 \\ & x_1, x_2 \geq 0, \text{ integer} \end{aligned}$$

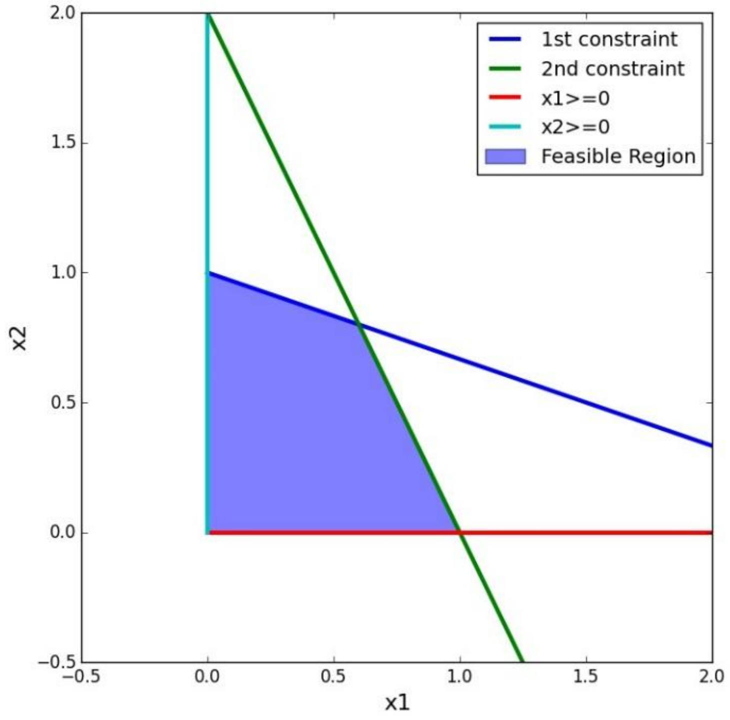


Example: Chvátal-Gomory Cut

- Suppose we have the following system:

$$\begin{aligned} \text{max} \quad & 5x_1 + 4x_2 \\ \text{subject to} \quad & x_1 + 3x_2 \leq 3 \\ & 2x_1 + x_2 \leq 2 \\ & x_1, x_2 \geq 0, \ \text{integer} \end{aligned}$$

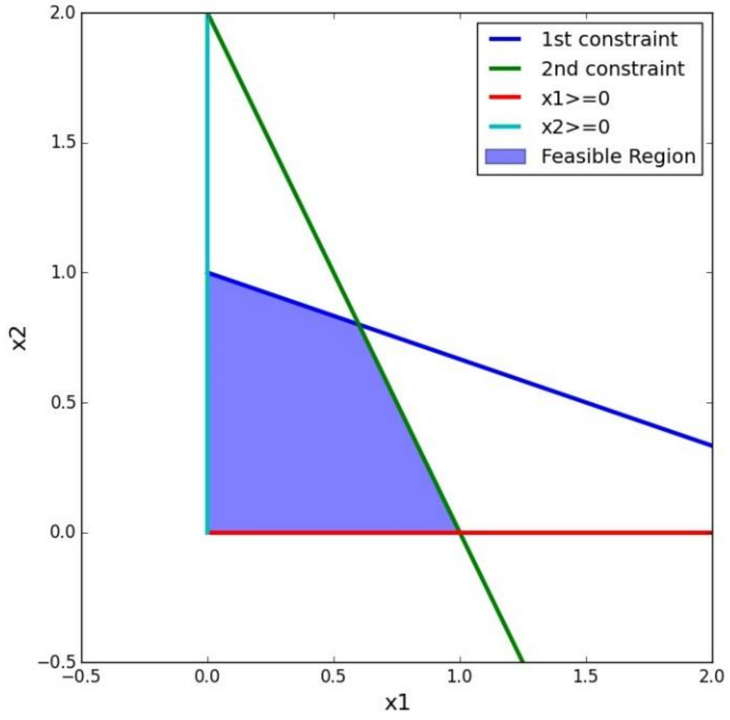
- Here are the important components
  - In matrix / vector form: $A = \begin{bmatrix} 1 & 3 \\ 2 & 1 \end{bmatrix}$ $b = \begin{bmatrix} 3 \\ 2 \end{bmatrix}$



Example: Chvátal-Gomory Cut

- Suppose we have the following system:

$$\begin{aligned} \text{max} \quad & 5x_1 + 4x_2 \\ \text{subject to} \quad & x_1 + 3x_2 \leq 3 \\ & 2x_1 + x_2 \leq 2 \\ & x_1, x_2 \geq 0, \text{ integer} \end{aligned}$$



- Here are the important components
  - In matrix / vector form: $A = \begin{bmatrix} 1 & 3 \\ 2 & 1 \end{bmatrix}$ $b = \begin{bmatrix} 3 \\ 2 \end{bmatrix}$

- Now suppose we use this (arbitrary) vector: $u^\top = \left[\frac{1}{5}, \frac{2}{5}\right]$

Example: Chvátal-Gomory Cut

- The operation $\lfloor u^\top A \rfloor$ yields $\lfloor 1, 1 \rfloor = [1, 1]$
- The operation $\lfloor u^\top b \rfloor$ yields $\lfloor 1.4 \rfloor = 1$

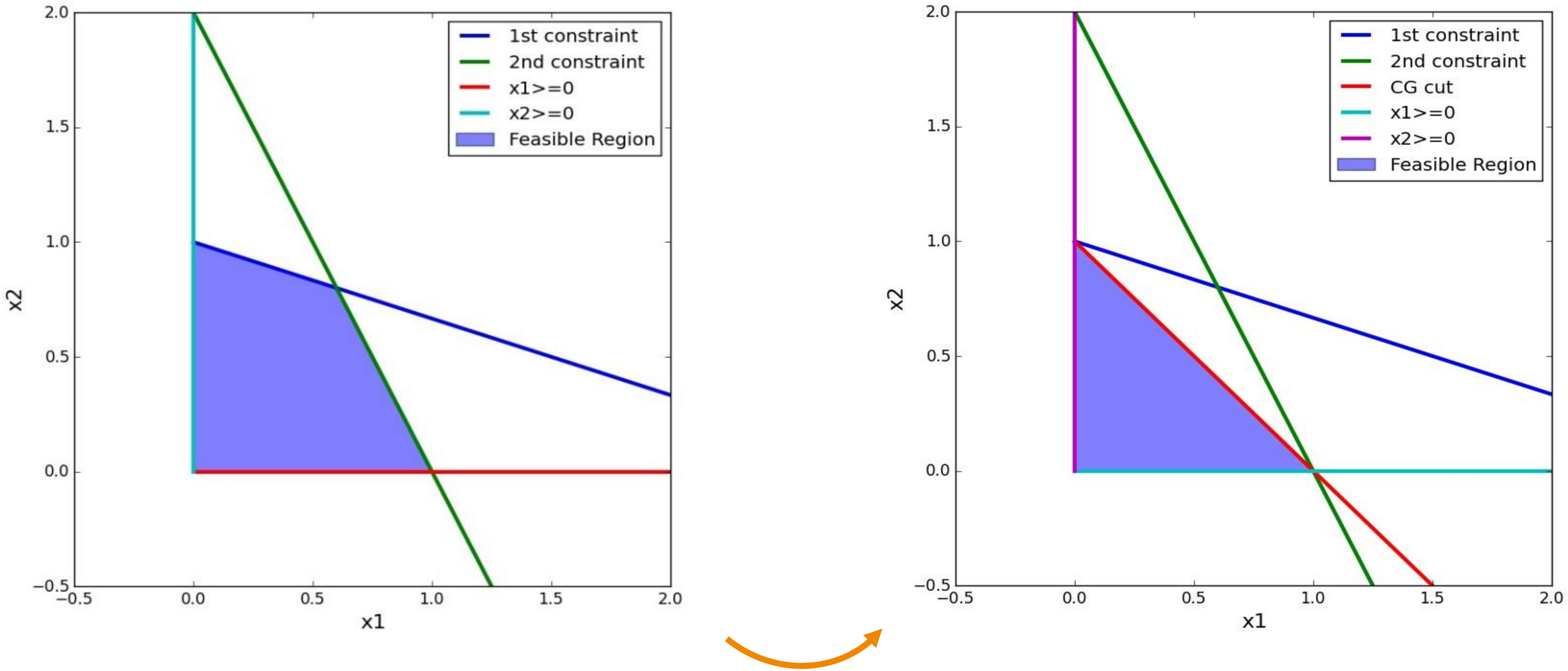
Example: Chvátal-Gomory Cut

- The operation $\lfloor u^\top A \rfloor$ yields $\lfloor 1, 1 \rfloor = [1, 1]$
- The operation $\lfloor u^\top b \rfloor$ yields $\lfloor 1.4 \rfloor = 1$

So, this generates the new Chvátal-Gomory (CG) cut: $x_1 + x_2 \leq 1$

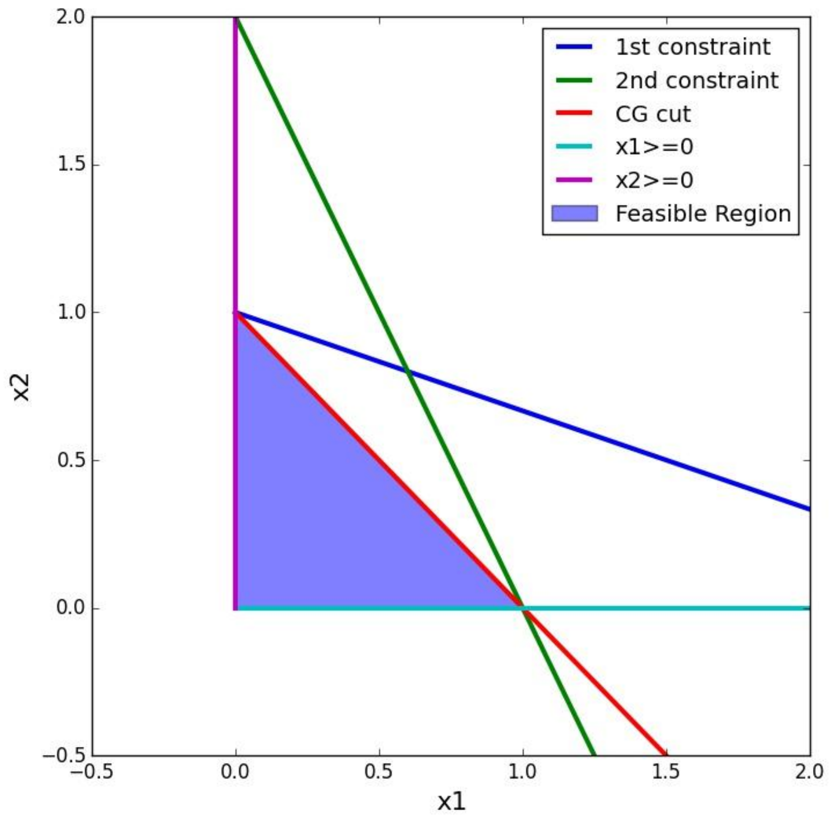
Example: Chvátal-Gomory Cut

So, this generates the new Chvátal-Gomory (CG) cut: $x_1 + x_2 \leq 1$



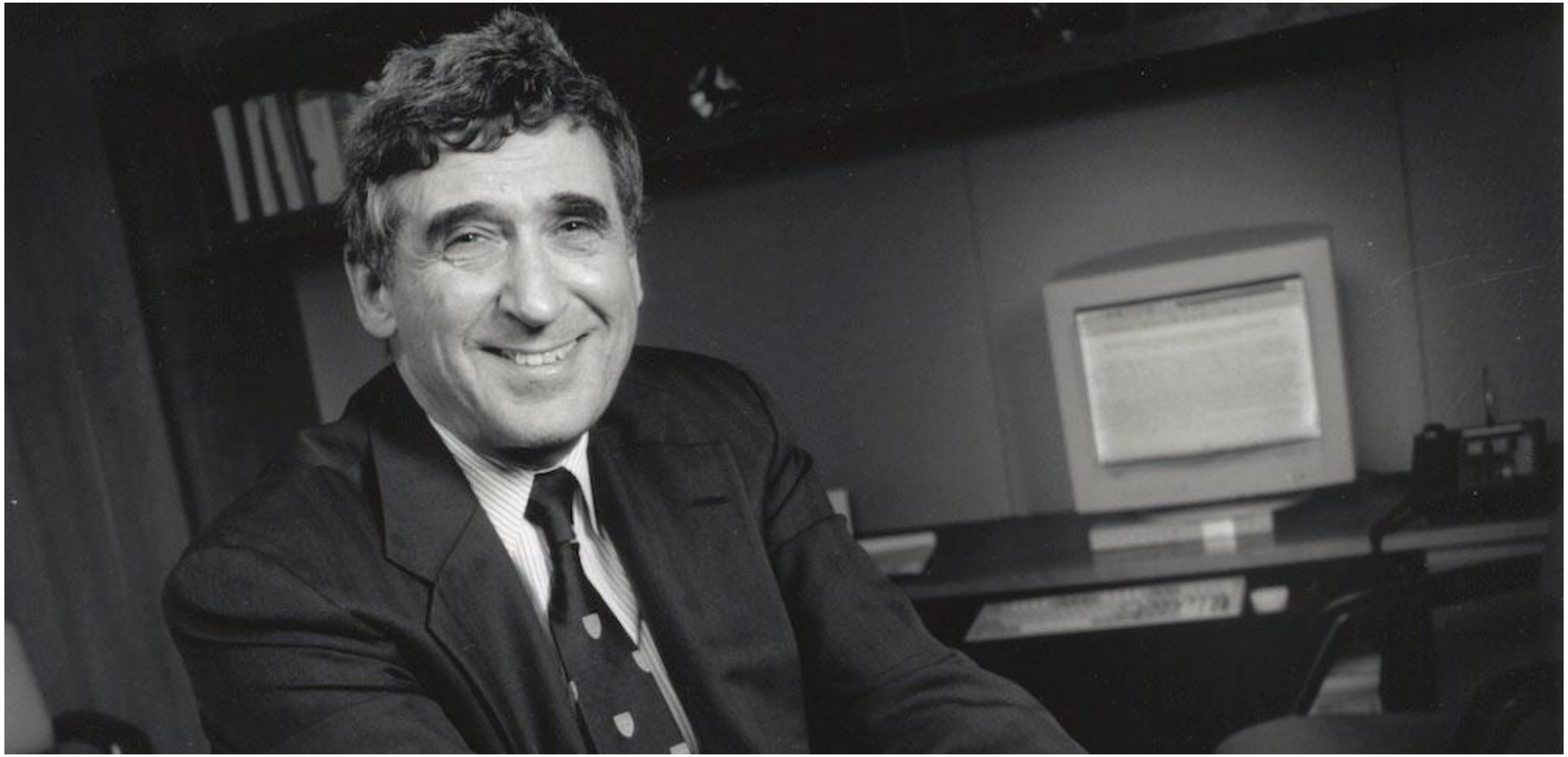
Example: Chvátal-Gomory Cut

- In this case, this operation yields the convex hull of feasible integer solutions
- In general, it's not obvious how to choose the vector u
- Can iteratively apply multiple CG cuts to remove fractional space



Ralph Gomory And IP

https://www.youtube.com/watch?v=OesVp4HIqps



Ralph Gomory. Source: *The Heinz Awards*. heinzawards.net (accessed August 7, 2025)

Cutting Plane Approach In A Nutshell

Another approach to solving integer programs is the cutting plane approach

- Pioneered by Ralph Gomory in the late 1950's

Cutting Plane Approach In A Nutshell

Another approach to solving integer programs is the cutting plane approach

- Pioneered by Ralph Gomory in the late 1950's

Just a brief overview of the main idea:

- First, solve the LP relaxation
- If the solution is non-integer, generate some kind of **valid inequality** that: Cuts off the fractional solution, and does not cut off any integer solution
- Repeat process until an integer solution is found

Cutting Plane Approach In A Nutshell

Another approach to solving integer programs is the cutting plane approach

- Pioneered by Ralph Gomory in the late 1950's

Just a brief overview of the main idea:

- First, solve the LP relaxation
- If the solution is non-integer, generate some kind of **valid inequality** that: Cuts off the fractional solution, and does not cut off any integer solution
- Repeat process until an integer solution is found

Sometimes valid inequalities, or cuts, are added dynamically during the branch and bound process: this is known as the **branch-and-cut algorithm**

Implementation of Algorithms Using Gurobi

Example: A Clothing Company

A Clothing Company is capable of manufacturing three types of clothing: shirts, shorts, and pants.

The manufacture of each type of clothing requires that company have the appropriate type of machinery available. The machinery needed to manufacture each type of clothing must be rented at the following rates:

- shirt machinery: $200 per week
- shorts machinery: $150 per week
- pants machinery: $100 per week

It also requires the amounts of cloth and labor as shown in Table 1. Each week, 150 hours of labor and 160 sq yd of cloth are available. The variable unit cost and selling price for each type of clothing are shown in Table 2. Formulate an IP whose solution will maximize company's weekly profits.

Resource Requirements

Clothing Type	Labor (Hours)	Cloth (Square Yards)
Shirt	3	4
Shorts	2	3
Pants	6	4

Revenue and Cost Information

Clothing Type	Sales Price ($)	Variable Cost ($)
Shirt	12	6
Shorts	8	4
Pants	15	8

Example: A Clothing Company

Define the decision variables:

x_1 = number of shirts produced each week

x_2 = number of shorts produced each week

x_3 = number of pants produced each week

$$y_1 = \begin{cases} 1 & \text{if any shirts are manufactured} \\ 0 & \text{otherwise} \end{cases}$$

$$y_2 = \begin{cases} 1 & \text{if any shorts are manufactured} \\ 0 & \text{otherwise} \end{cases}$$

$$y_3 = \begin{cases} 1 & \text{if any pants are manufactured} \\ 0 & \text{otherwise} \end{cases}$$

In short, if $x_j > 0$, then $y_j = 1$, and if $x_j = 0$, then $y_j = 0$.

Example: A Clothing Company

Define the objective function:

Company's Weekly Profits:

(weekly sales revenue) - (weekly variable costs) - (weekly costs of renting machinery)

$$\begin{aligned}\text{Weekly profit} &= (12x_1 + 8x_2 + 15x_3) - (6x_1 + 4x_2 + 8x_3) \\ &\quad - (200y_1 + 150y_2 + 100y_3) \\ &= 6x_1 + 4x_2 + 7x_3 - 200y_1 - 150y_2 - 100y_3\end{aligned}$$

Maximize $z = 6x_1 + 4x_2 + 7x_3 - 200y_1 - 150y_2 - 100y_3$

Example: A Clothing Company

Then the IP becomes:

$$
\begin{aligned}
\text{max } z &= 6x_1 + 4x_2 + 7x_3 - 200y_1 - 150y_2 - 100y_3 \\
\text{s.t.} \quad & 3x_1 + 2x_2 + 6x_3 \le 150 \\
& 4x_1 + 3x_2 + 4x_3 \le 160 \\
& x_1, x_2, x_3 \ge 0;\ x_1, x_2, x_3 \text{ integer} \\
& y_1, y_2, y_3 = 0 \text{ or } 1 \\
& x_1 \le M_1 y_1 \\
& x_2 \le M_2 y_2 \\
& x_3 \le M_3 y_3
\end{aligned}
$$